



Hochschule für Angewandte Wissenschaften Hamburg
Hamburg University of Applied Sciences

Computational Intelligence

Prof. Dr.-Ing. Sabine Schumann

Fakultät Informatik und Digitale Gesellschaft

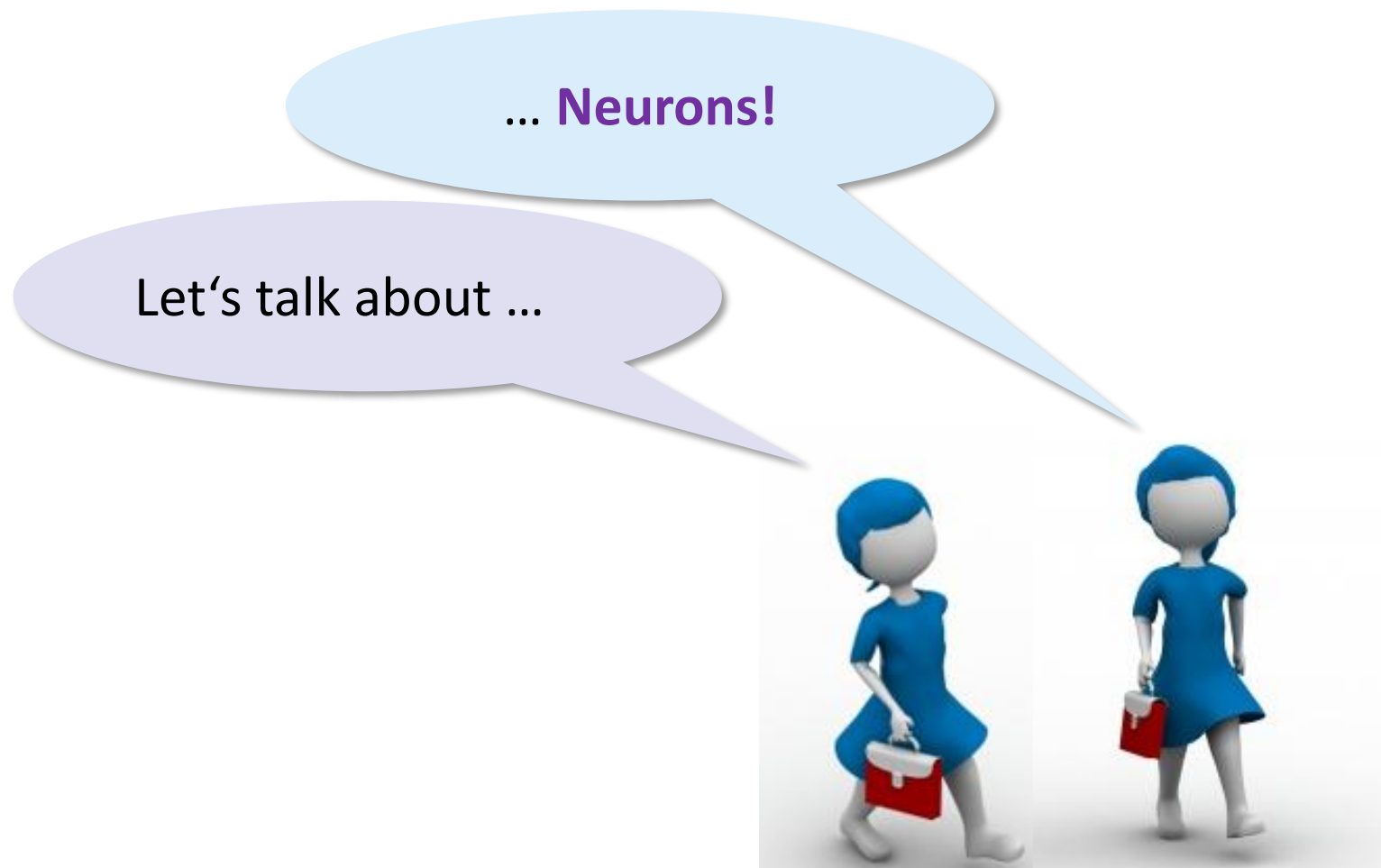
Künstliche Neuronale Netze

Murphy ...

Nichts endet so wie geplant.

Seay's Gesetz,

Arthur Bloch: Murphy's Gesetze in einem Band, 1985



Bilder: Freedigitalphotos.net: Renjith Krishnan, ID-10082483 und ID-10086282

Was erwartet Dich in diesem Kapitel?

Worüber reden wir hier überhaupt?

Entstehungsgeschichte

Neuron / Perceptron

Netze

Lernen ...

Tools

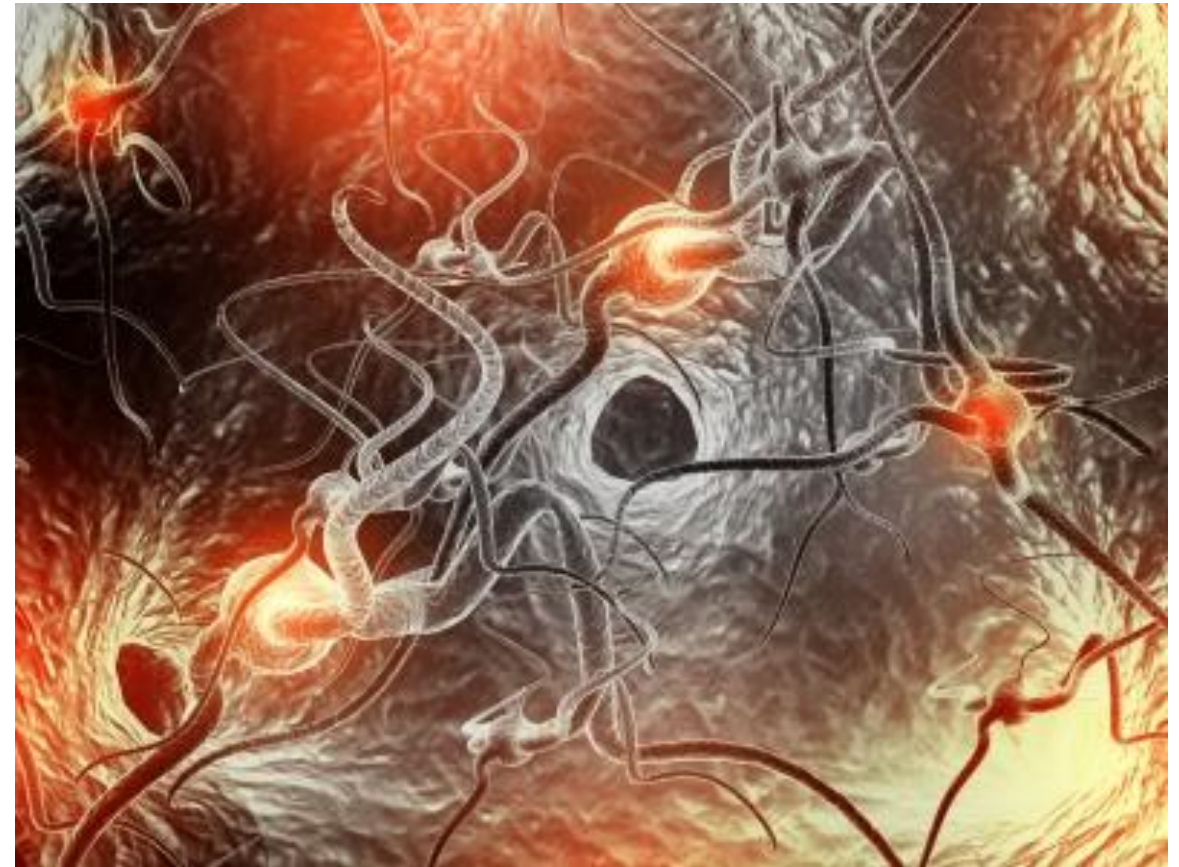


Bild: Freedigitalphotos.net: Renjith Krishnan, ID-100122589

(Künstliche) Neuronale Netze – Wieso Weshalb Warum?

(Neuro)Biologie, (Neuro)Physiologie, Psychologie

- Ausnutzung der Ähnlichkeit zu echten (biologischen) neuronalen Netzen,
- Modellierung zum Verständnis der Arbeitsweise von Nerven und Gehirn durch Simulation.

Informatik, Ingenieurwissenschaften, Wirtschaft

- Nachahmen der menschlichen Wahrnehmung und Verarbeitung,
- Lösen von Lern- und Anpassungsproblemen,
- Lösen von Vorhersage- und Optimierungsproblemen.

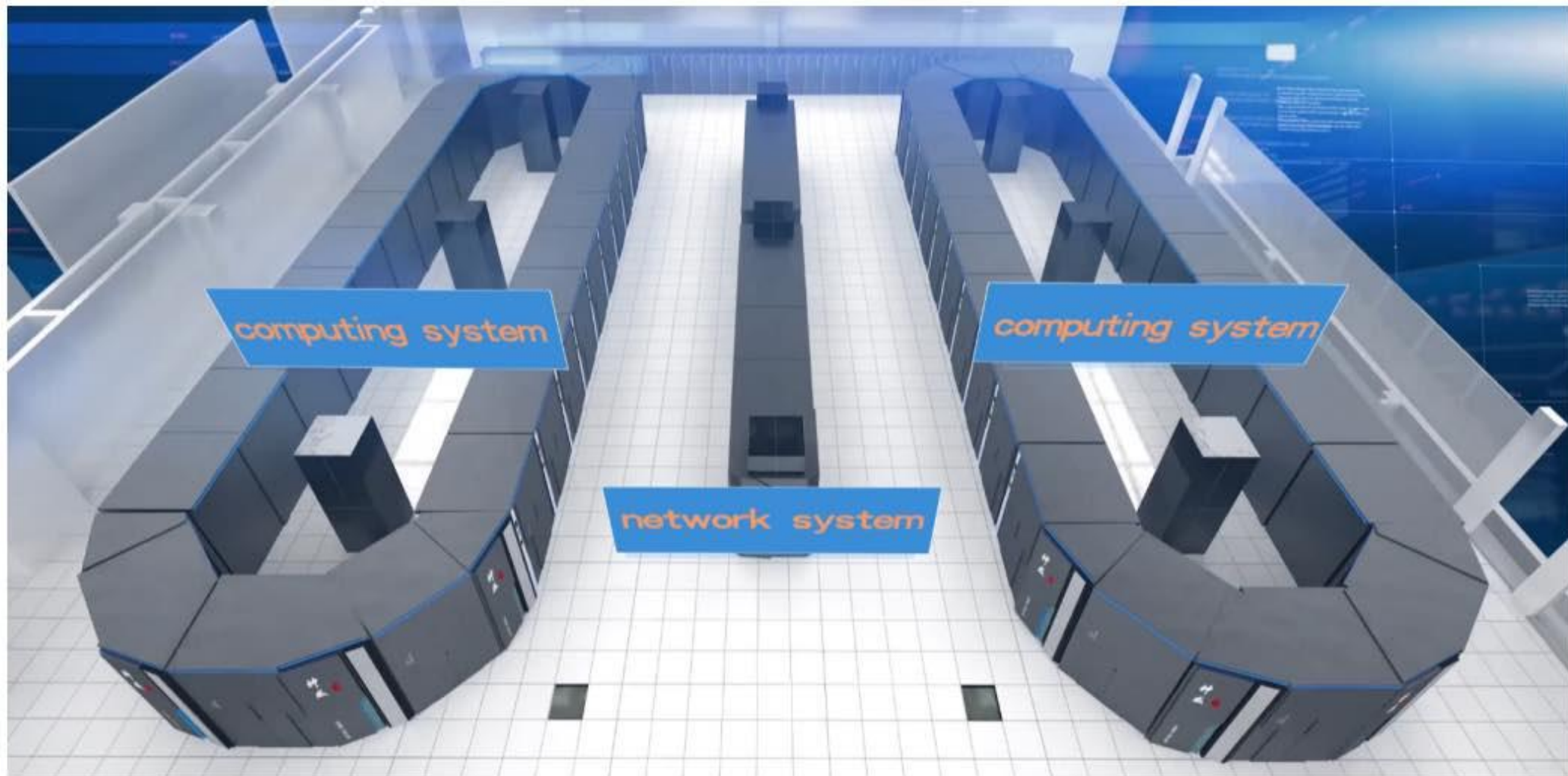
Physik, Chemie

- Nutzung neuronaler Netze, um physikalische Phänomene zu beschreiben.

Anwendungen

- Frühwarnsysteme,
- Optimierung,
- Bildverarbeitung und Mustererkennung
 - Data Mining,
 - Schrift- und Spracherkennung,
- Sprachgenerierung,
- Klangsynthese,
- Datenvorverarbeitung,
- Vergleichsmodelle im sozialen bzw. betriebswirtschaftlichen Umfeld,
- Zeitreihenanalyse (Aktien, Wetter, Verkehr, ...),
- Robotik, virtuelle Agenten, Spiel-Simulatoren,
- ...

Sunway TaihuLight 2016



Bildquelle: <http://insidehpc.com/wp-content/uploads/2016/06/sunway-1.jpg>

Rechner versus Gehirn (1)

	Computer	Mensch / Gehirn
Geschwindigkeit	93 Petaflops / s	10.000 Teraflops / s
Gewicht	>100 Tonnen	ca. 1,4 kg
Memory	1.310.720 GB	10 ¹¹ Neuronen 10 ¹⁴ Synapsen
Cores	10.649.600	10 ¹¹ Neuronen (100 Milliarden)
Energiebedarf pro Tag	73.983.041 KJ	ca. 9.000 KJ
Anschaffungskosten	273.000.000 \$	0

Quellen: Neuronale Netze, Rudolf Kruse und https://de.wikipedia.org/wiki/Sunway_TaihuLight und Wolfgang Wahlster, DFKI, 2016

Rechner versus Gehirn (2)

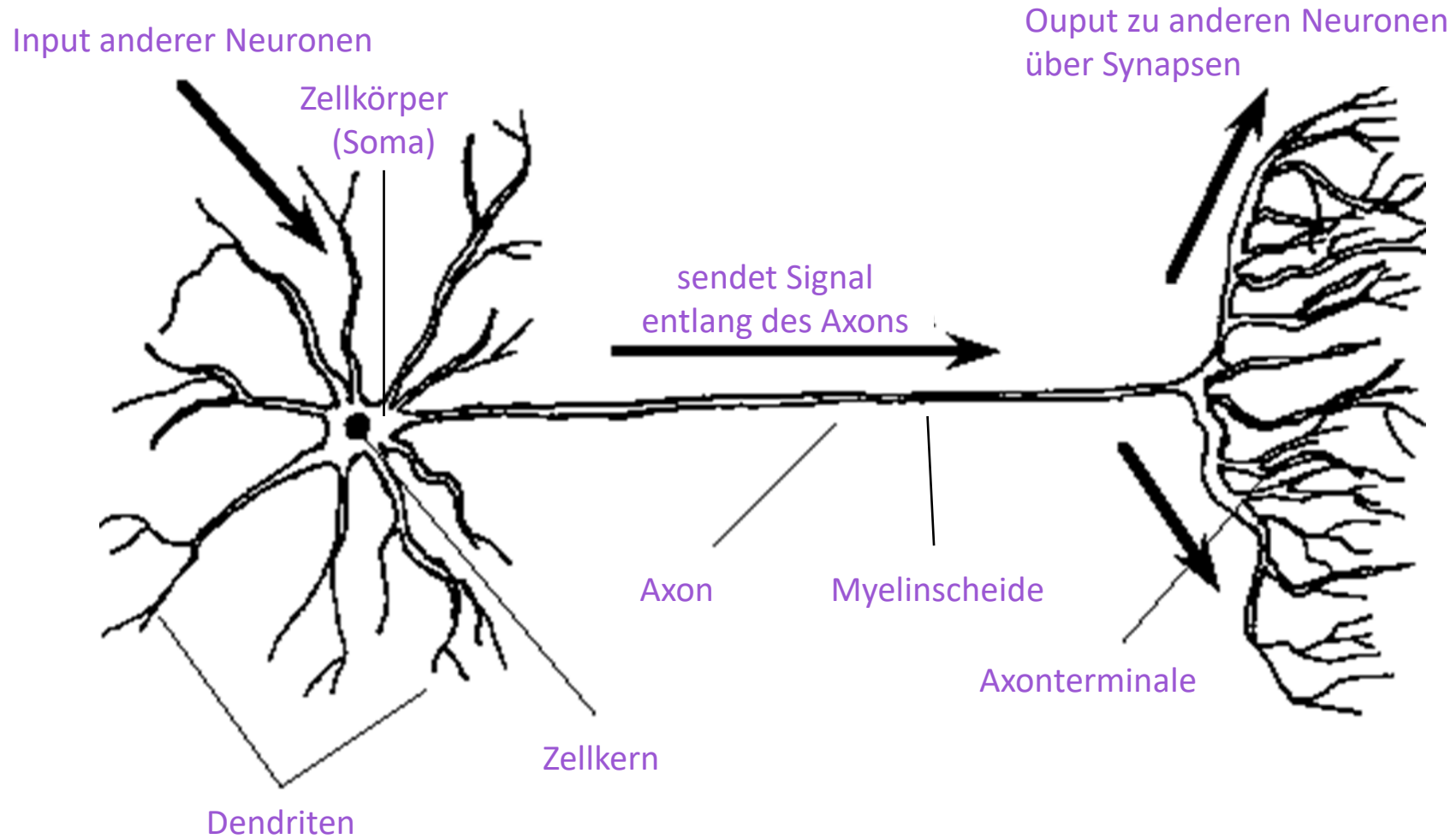
Vorteile Neuronaler Netze:

- Hohe Verarbeitungsgeschwindigkeit durch massive Parallelität,
- Funktionstüchtigkeit selbst bei Ausfall von Teilen des Netzes (Fehlertoleranz),
- Langsamer Funktionsausfall bei fortschreitenden Ausfällen von Neuronen (*graceful degradation*),
- gut geeignet für induktives Lernen.

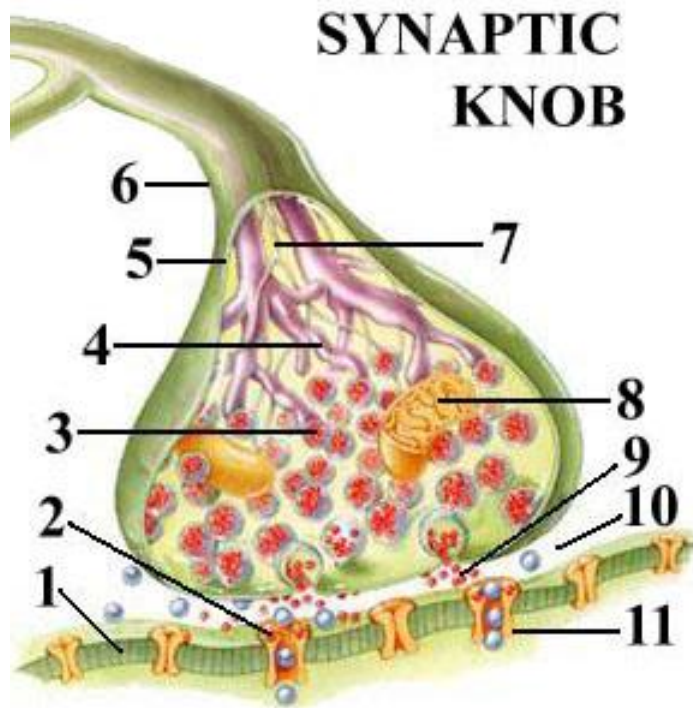


Daher sinnvoll, diese Vorteile natürlicher neuronaler Netze künstlich nachzuahmen.

biologisches Neuron



Synaptischer Knoten

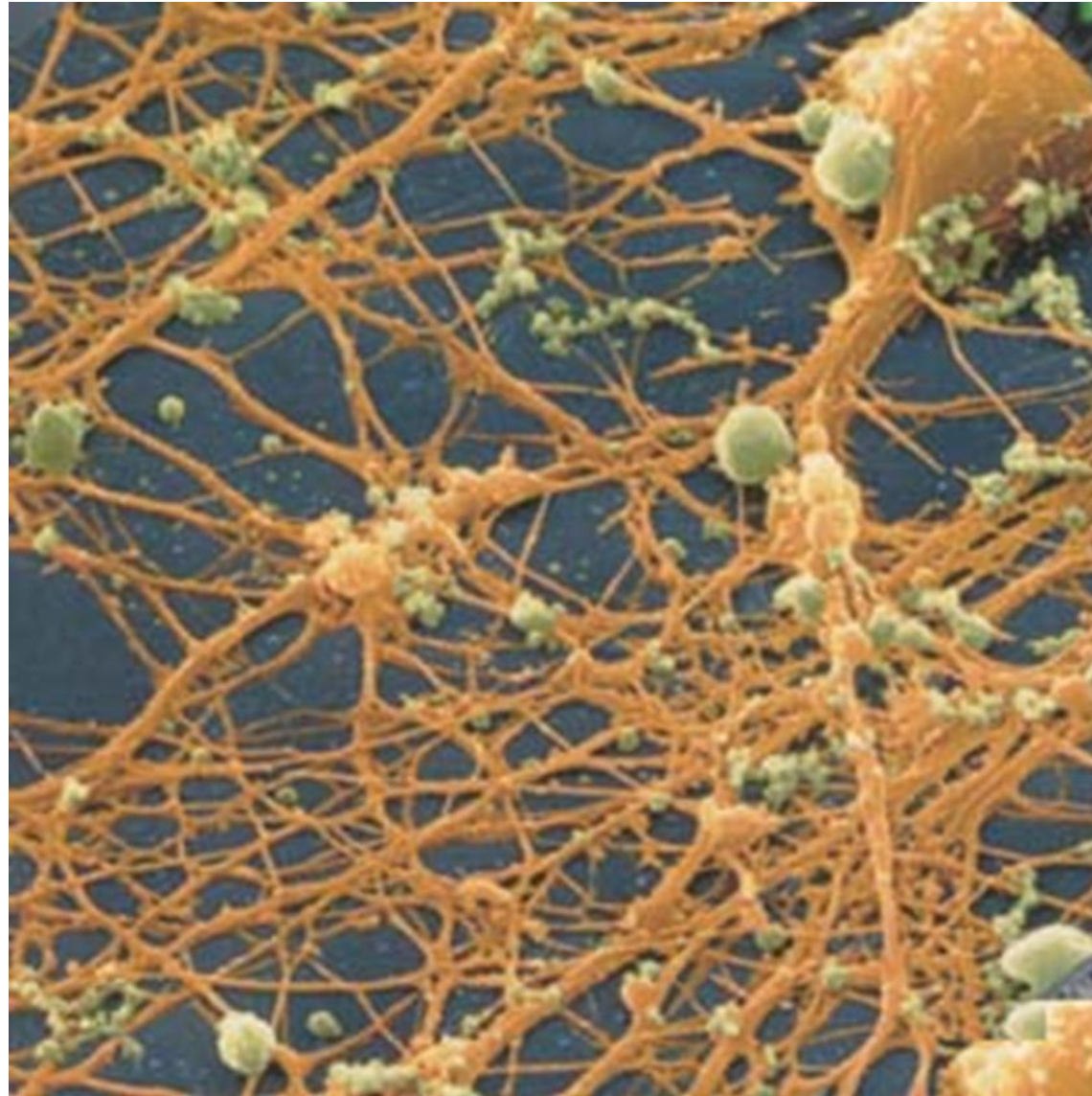


1. Target cell membrane
2. Receptor sites
3. Synaptic vesicles
4. Microtubules
5. Cell membrane
6. Axon terminal fiber
7. Neurofilaments
8. Mitochondrion
9. Neurotransmitter molecules
10. Synaptic cleft
11. Membrane channels

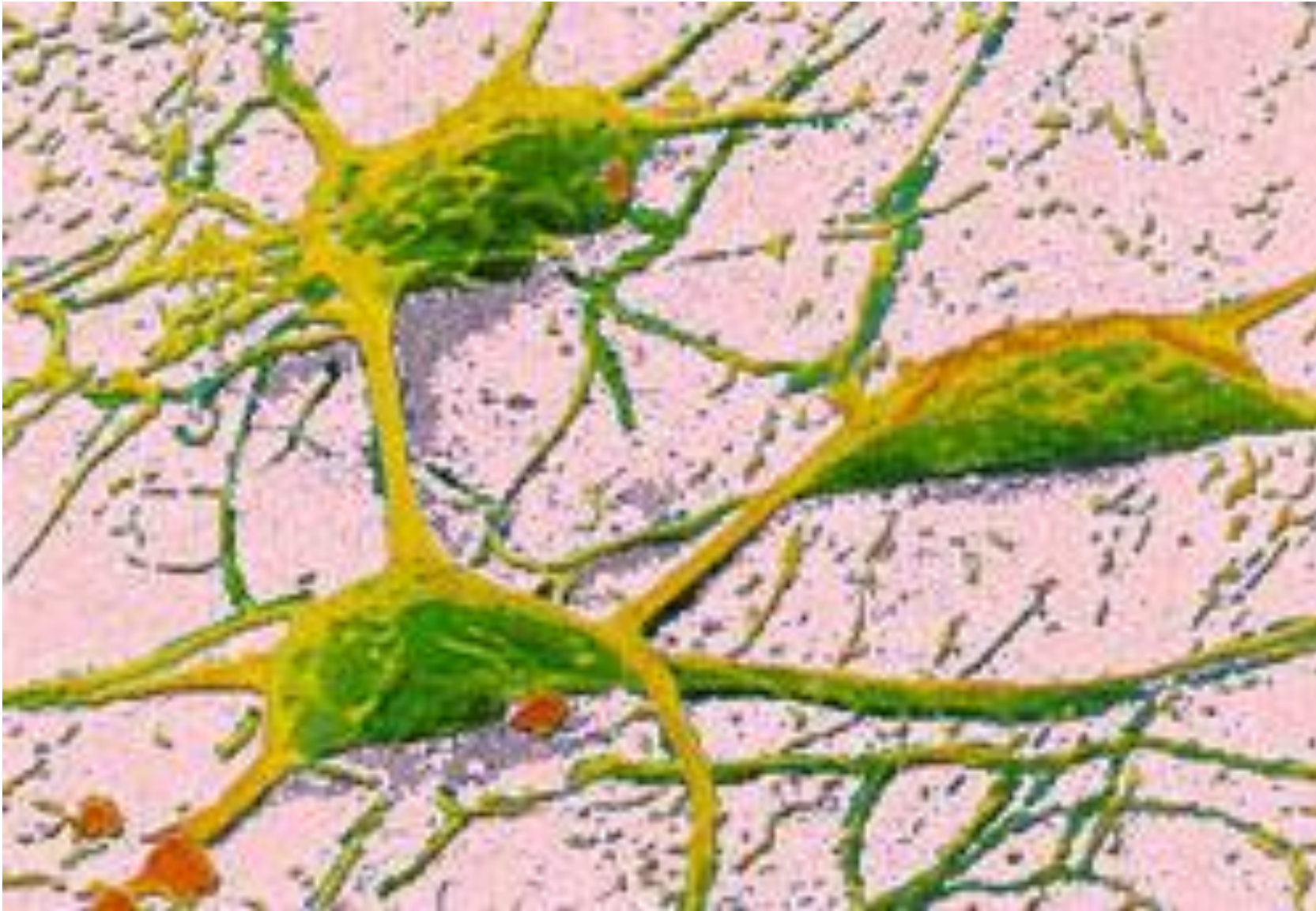
Wie funktioniert?

- Das ankommende Aktionspotential verursacht die Ausschüttung chemischer Botenstoffe (Neurotransmitter) (9) in den synaptischen Spalt (10).
- Das bewirkt Veränderungen an der Membran des Empfängerendriten. Es werden bestimmte Ionen-Kanäle (11) geöffnet, so dass
 - Natrium-Ionen hineinströmen oder
 - Kalium-Ionen herausströmen.
- Damit wird eine Veränderung der Polarisierung erreicht (das Innere ist typischerweise 70mV negativer als die Außenseite).
- Einstrom von Natrium-Ionen bewirkt Abnahme der Potentialdifferenz (erregende Synapsen).
- Ausstrom von Kalium-Ionen bewirkt Zunahme der Potentialdifferenz (hemmende Synapse).
- Wenn genügend anregende Information vorhanden ist, wird das Axon depolarisiert (bzw. bei hemmenden Synapsen hyperpolarisiert).
- Das resultierende Aktionspotential pflanzt sich entlang des Axons fort, die Geschwindigkeit hängt von der Bedeckung mit Myelin ab.
- Wenn das Aktionspotential die Synapsenenden erreicht, löst es die Abgabe von Neurotransmittern aus.

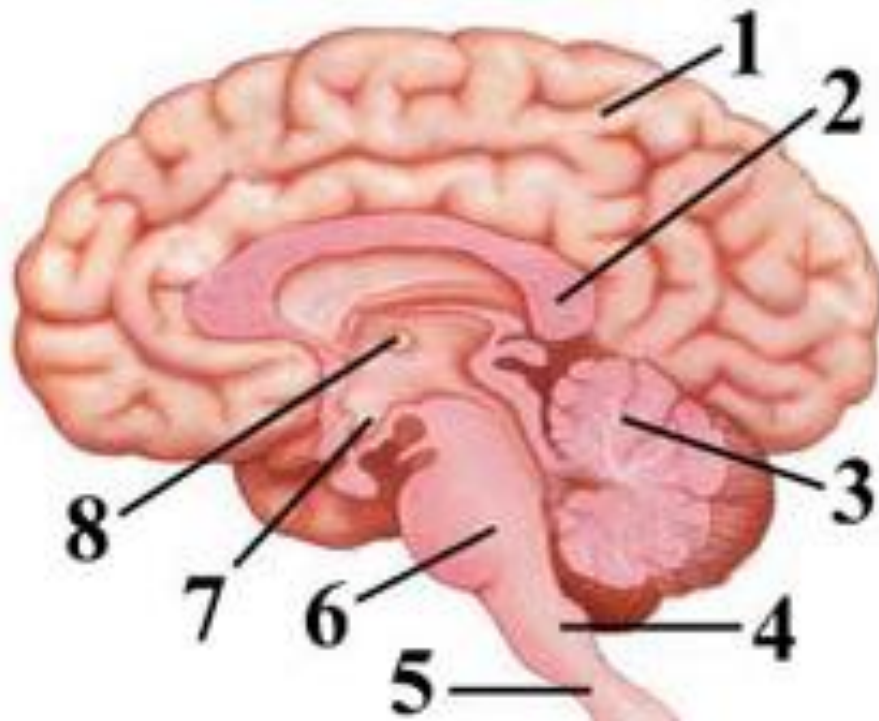
Neuronales Netzwerk



Neuronales Netzwerk des Zerebralen Kortex

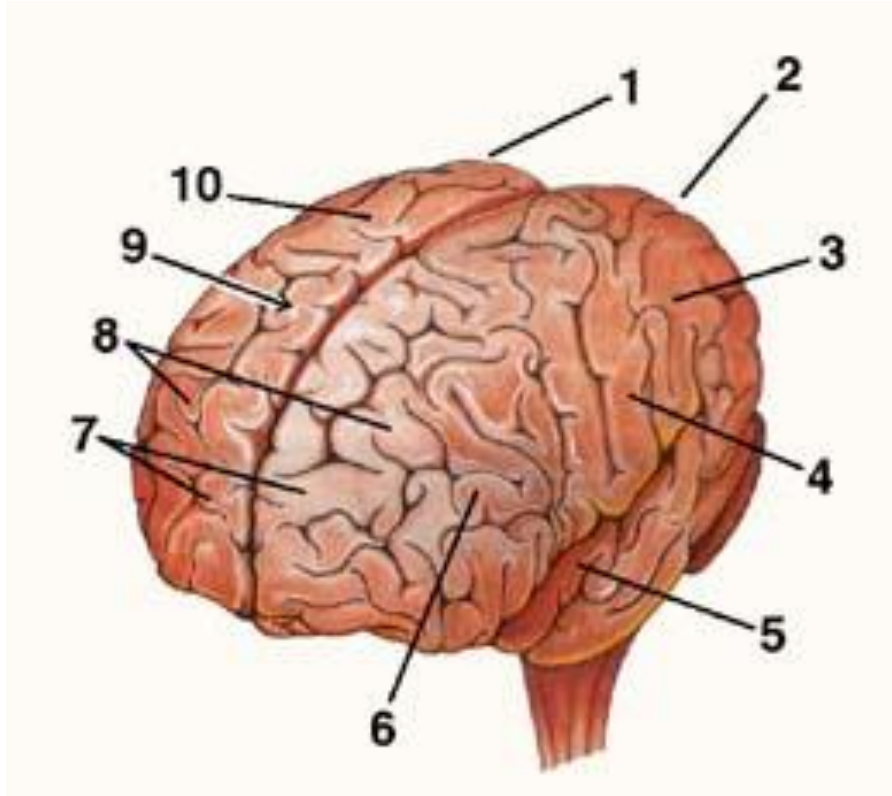


Menschliches Gehirn



- 1. Cerebrum
- 2. Corpus callosum
- 3. Cerebellum
- 4. Reticular-activating system
- 5. Spinal cord
- 6. Brain stem
- 7. Hypothalamus
- 8. Thalamus

„Landkarte“ des menschlichen Gehirns



1. Right cerebral hemisphere
2. Left cerebral hemisphere
3. Analytical reasoning
4. Understanding spoken language
5. Face recognition
6. Speech production
7. Abstract reasoning ---
in frontal lobes
 - Mathematical calculations
8. Working memory
9. Anticipation --- inferior
10. Spatial analysis

Neuronales Netzwerk

von hier aus nur ein kurzer Sprung zum

Künstlichen Neuronalen Netzwerk

Was erwartet Dich in diesem Kapitel?

Worüber reden wir hier überhaupt?

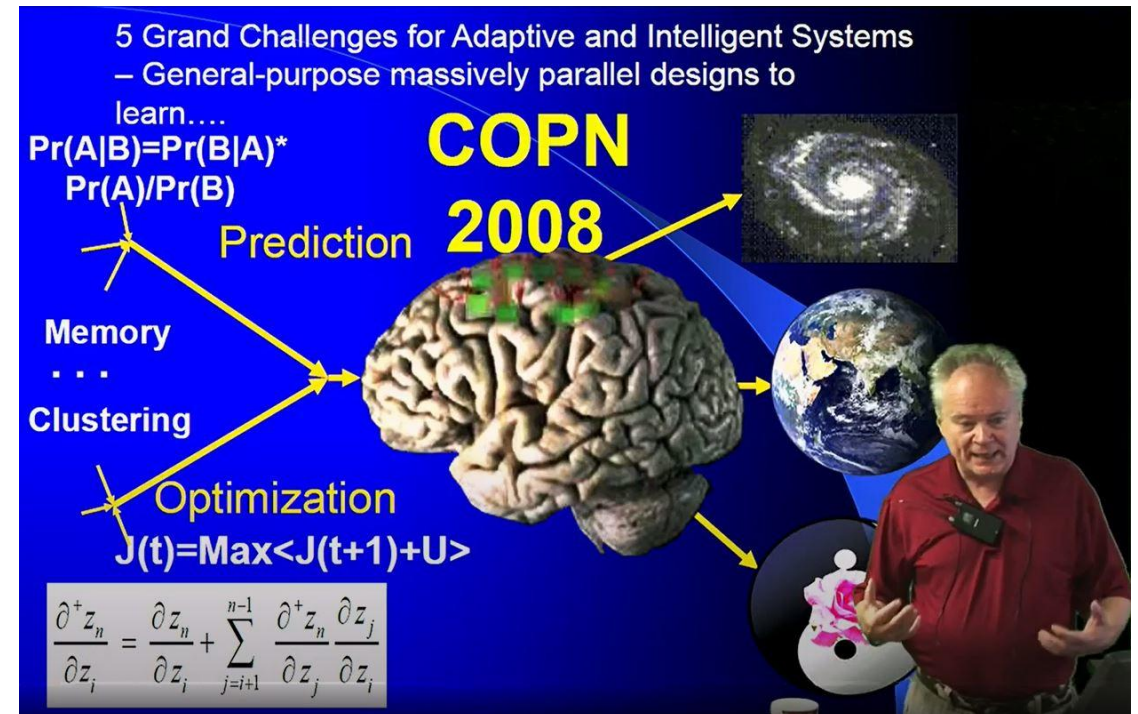
Entstehungsgeschichte

Neuron / Perceptron

Netze

Lernen ...

Tools



Bildquelle: <https://www.youtube.com/watch?v=6q1HqRd9MnA>

Blick in die Geschichte (1)



Warren Sturgis McCulloch
1898-1969

1943: Warren S. McCulloch & Walter Pitts

„A logical calculus of the ideas immanent in nervous activity“

Beschreibung einer Art neurologischen Netzes, das jede arithmetische und logische Operation berechnen kann

(☞ [McCulloch-Pitts Neuronen](#))



Walter Pitts
1923-1969

1949: Donald O. Hebb

„The Organization of Behaviour“

Die sogenannte ☞ [Hebbsche Lernregel](#)

ist bis heute Basis fast aller neuronaler Lernverfahren



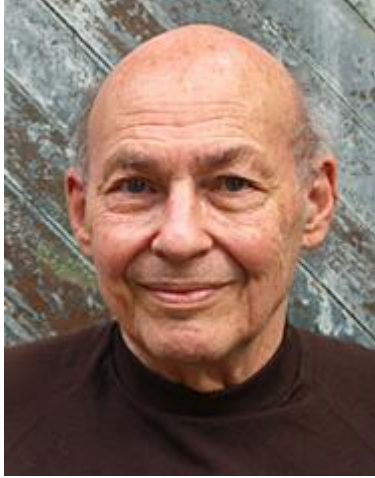
Donald Olding Hebb
1904-1985

Bildquellen:

<https://www.techfak.uni-bielefeld.de/ags/wbski/lehre/digiSA/journalclubki/ws1314/>

http://timerime.com/upload/resized/156144/1757253/resized_image2_3249526627761d83c72e4e90adbd5b58.jpg

Blick in die Geschichte (2)



Marvin Minsky
1927 - 2016

1951: Marvin Minsky und Dean Edmonds

- erster bekannter Neurocomputer *SNARC* (Dissertation)
- **S**tochastic **N**eural **A**nalog **R**einforcement **C**alculator
- Simulation einer Maus im Labyrinth
- wurde nie praktisch umgesetzt

Bildquelle: http://www.cse.unsw.edu.au/~billw/cs9414/notes/kr/frames/2006_marvin_minsky.jpg

Blick in die Geschichte (3)

1957/58:

erster erfolgreicher Neurocomputer *Mark I perceptron*
von Frank Rosenblatt, Charles Wightman u.a. (MIT)

1962:

Frank Rosenblatt

„*Principles of Neurodynamic: Perceptrons and the Theory of Brain Mechanisms*“
verschiedene Varianten des Perceptrons und [Perceptron-Konvergenz-Theorem](#)



Frank Rosenblatt
1928 -1971

Perzeption

Wahrnehmung als erste Stufe der Erkenntnis

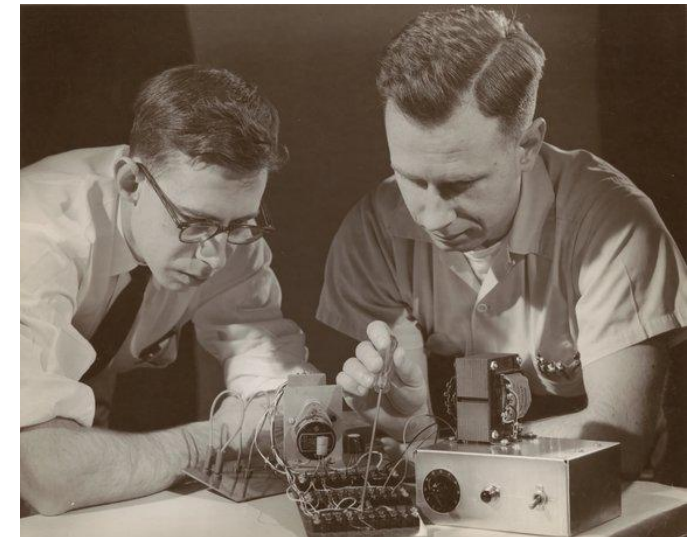
in der Psychologie der Vorgang der (sinnlichen) Wahrnehmung eines Gegenstandes ohne bewusstes Erfassen und Identifizieren des Wahrgenommenen

Meyers Neues Lexikon

Bildquellen:

http://www.informatics4kids.de/images/ik4/softcomputing/neural_network/rosenblatt.jpg

<http://csis.pace.edu/~ctappert/srd2011/photos/Rosenblatt-CAL1958.jpg>



Blick in die Geschichte (4)

Minsky und Rosenblatt waren erbitterte Widersacher.

1969:

Marvin Minsky und Seymour Papert: „*Perceptrons*“
mathematischer Nachweis, dass das Perceptron-Konzept
nicht in der Lage ist, XOR abzubilden.

Demzufolge sei es ein **research dead end**.

- Aus diesem Grund wurden fast 15 Jahre keine Forschungsgelder für Neuronale Netze genehmigt.
- Spätere Algorithmen (bspw. Backpropagation-Netz) beheben dieses Defizit.
- Ironischerweise arbeitete Minsky in den 80er Jahren selbst wieder auf dem Gebiet der neuronalen Netze und seine Forschungen basierten auf den Ergebnissen von Frank Rosenblatt.



Seymour Papert
1928 - 2016

Blick in die Geschichte (5)

1960:

Bernard Widrow und Marcian E. Hoff

- ADALINE (**AD**aptive **L**inear **NE**uron)
- erste kommerzielle Verbreitung in Analog-Telefonen zur Echtzeit-Echofilterung
- binäres Schwellenwert-Neuron
- Widrow-Hoff-Regel
(LMS-Algorithmus bzw. ➞ [Delta-Regel](#))

1972:

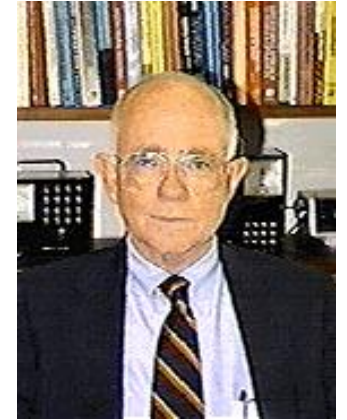
Teuvo Kohonen

- Erfinder der Self-Organizing Maps (**1982**)
- Assoziativspeicher
- Selbstorganisation
- Mustererkennung

Bildquellen:

http://www-isl.stanford.edu/people/widrow/Widrow_Bernard.gif

<http://itc.ua/img/ko/2009/05/small/034157.jpg>



Bernard Widrow
* 1929



Teuvo Kohonen
1934-2021

Blick in die Geschichte (6)



Christoph von der Malsburg
* 1942

1973:

Christoph von der Malsburg

benutzt ein nichtlineares Neuronenmodell
später: Gesichtserkennung

1982-1985:

John Hopfield

- Modell der binären Hopfield-Netze
- Lösung des Travelling Salesman Problems durch ein Hopfield-Netz



John Hopfield
* 1933

Bildquellen:

http://medien.filmreporter.de/images/thumbnails/10137_300.jpg

http://bulletin.swarthmore.edu/bulletin-issue-archive/wp-content/uploads/2009/October/BOX_hopfield_john.jpg

Blick in die Geschichte (7)

1980/1984::

Kunihiko Fukushima:

Neocognitron:

A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position

https://www.researchgate.net/publication/15829465_Neocognitron_A_self-organizing_neural_network_model_for_a_mechanism_of_pattern_recognition_unaffected_by_shift_in_position



Kunihiko Fukushima
*1936

Kunihiko Fukushima, Sei Miyake:

Neocognitron: A neural network model for a mechanism of visual pattern recognition

positions- und skalierungsinvariante Erkennung handgeschriebener Zeichen

https://www.researchgate.net/publication/16718287_A_neural_network_model_for_the_mechanism_of_feature-extraction_A_self-organizing_network_with_feedback_inhibition

Bildquelle: https://i1.rgstatic.net/ii/profile.image/AS%3A416260954312704%401476255753417_I/Kunihiko_Fukushima2.png

Blick in die Geschichte (8)

1974 / 1986:

1974 Paul Werbos entwickelt das Verfahren zur Backpropagation (Dissertation)

1986 David E. Rumelhart, Geoffrey E. Hinton, Ronald Williams können es in die Praxis umsetzen

Lernverfahren ➡ [Backpropagation of Error](#) (Fehlerrückführung, Verallgemeinerung der Delta-Regel)



Paul Werbos
* 1947



David Everett Rumelhart
1942-2011



Geoffrey Everest Hinton
* 1947

- explosionsartige Entwicklung seit 1986 („2. Generation“)
- aktuell „3. Generation“ : Spiking neural networks

Bildquellen: http://www.werbos.com/index_files/image006.jpg

http://www.psychologicalscience.org/redesign/wp-content/uploads/2011/11/Rumelhart_1.jpg

<https://royalsociety.org/~media/events/2015/05/breakthrough-science-technologies-machine-learning/machine-learning-hinton.jpg?w=500>

PlaNet

**„Ob Landschaftsbild, Auto oder Pferd:
Ein künstliches neuronales Netz erkennt besser als Menschen, wo ein Foto aufgenommen wurde.
Sogar seine Fehler scheinen sehr menschlich.“**



28.02.2016

<http://www.spiegel.de/netzwelt/web/planet-computerprogramm-erkennt-aufnahmeort-von-fotos-a-1079427.html>

Was erwartet Dich in diesem Kapitel?

Worüber reden wir hier überhaupt?

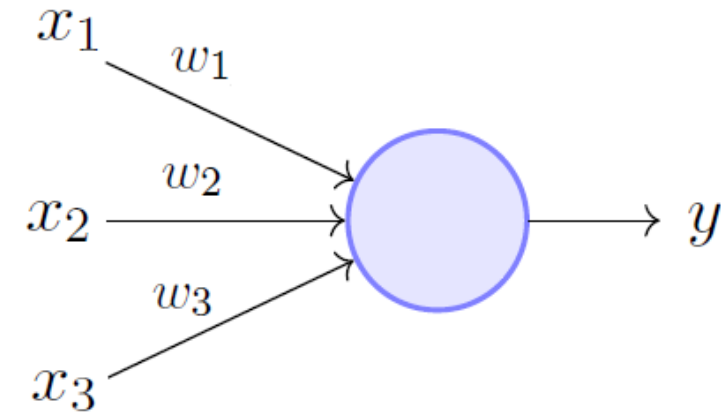
Entstehungsgeschichte

Neuron / Perceptron

Netze

Lernen ...

Tools



Bildquelle: <https://towardsdatascience.com/what-is-a-perceptron-210a50190c3b>

McCulloch-Pitts-Neuronen

basiert auf folgenden Annahmen:

- Ein Neuron ist ein binäres Schaltelement, welches entweder aktiv oder inaktiv ist.
- Jedes Neuron besitzt einen festen Schwellwert.
- Ein Neuron empfängt sowohl Eingaben von erregenden Synapsen, die alle das gleiche Gewicht haben, als auch von hemmenden Synapsen.
- Eine einzige hemmende Synapse verhindert die Aktivierung des Neurons.
- Falls keine hemmende Synapse aktiv ist, werden die erregenden Eingaben addiert. Bei der Überschreitung des Schwellenwertes wird das Neuron aktiv.



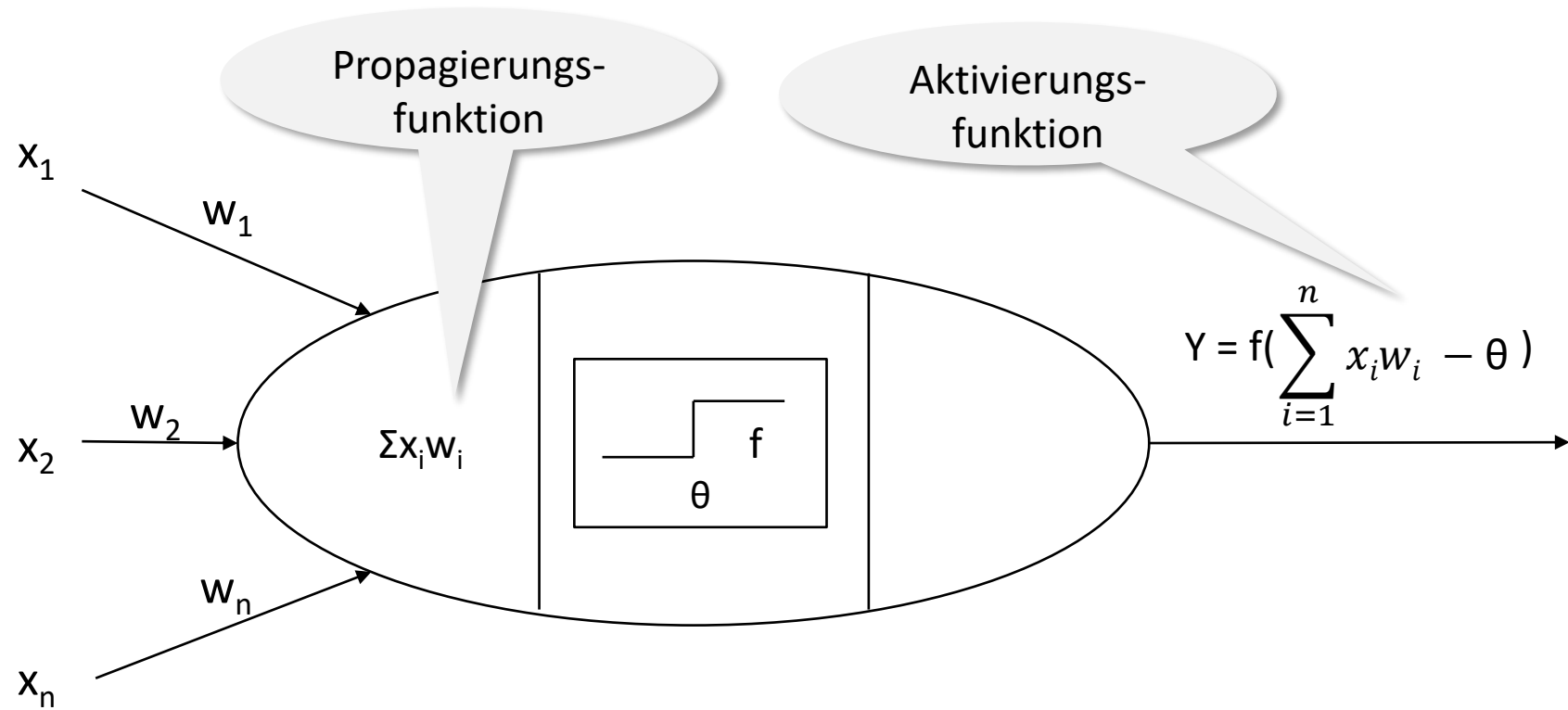
ist ein Spezialfall der Threshold Logic Unit (TLU) (Perceptron)

Threshold Logic Unit (1)

Threshold Logic Unit (TLU) (einfaches Perceptron):

- **Input** besteht nur aus den BOOLEschen Werten (0 und 1).
- **Gesamtinput** ergibt sich als gewichtete Summe der Inputwerte $x_1, \dots, x_n$
- **Aktivierung** des Neurons erfolgt über die **Aktivierungsfunktion** f , wenn der Gesamtinput größer oder gleich dem **Schwellwert** θ (threshold) ist
(f kann also als Funktion über der Differenz aus Gesamtinput und Schwellwert θ angesehen werden).
- **Output-Wert** y entsteht je nach Aktivierung
(ist demnach der Indikatorwert der Aktivierung).

Threshold Logic Unit (2a)

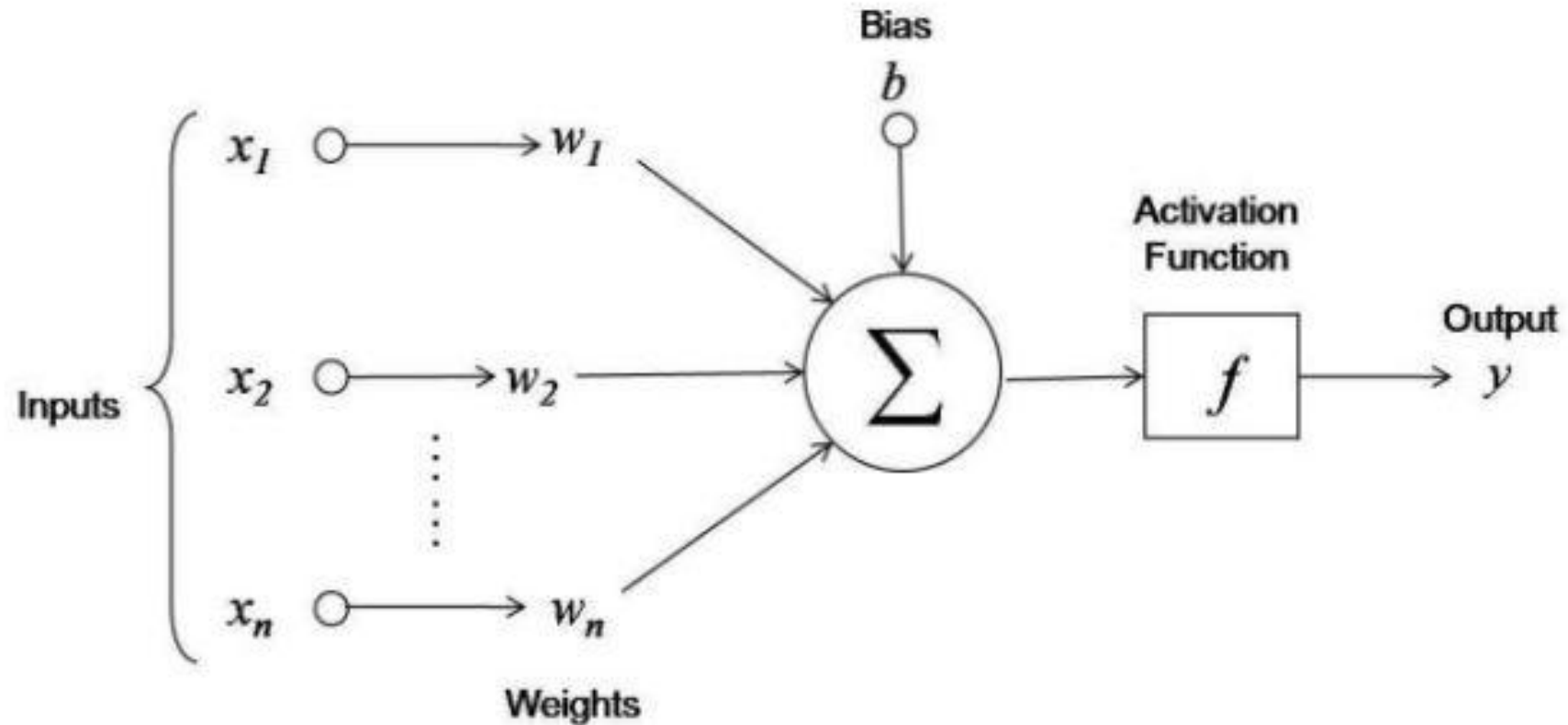


$$y=1 \text{ wenn } \sum_{i=1}^n x_i w_i \geq \theta$$

$y=0$ andernfalls

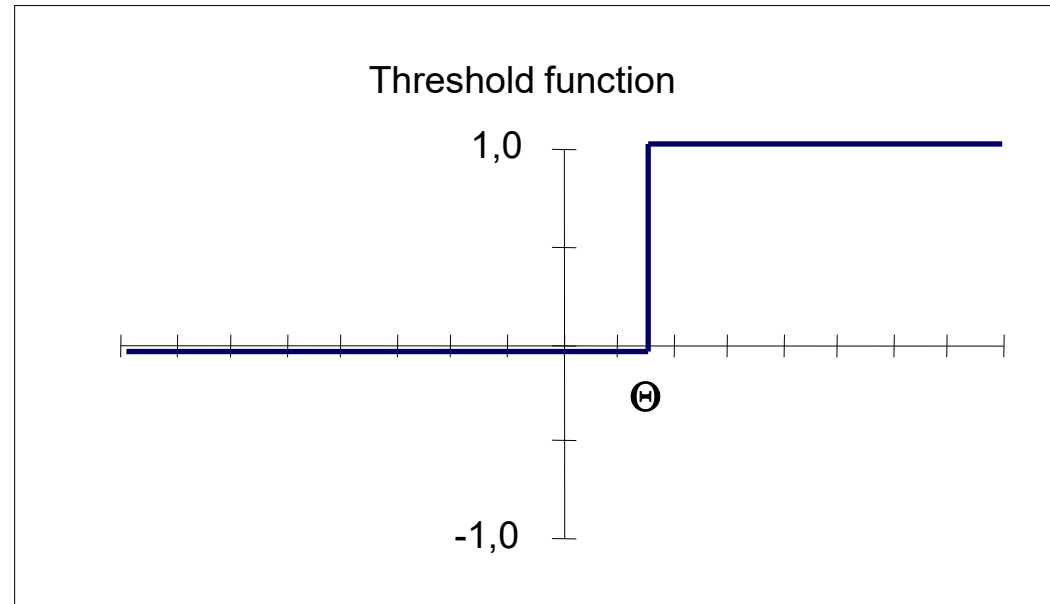
Threshold Logic Unit (2b)

Anders gezeichnet:



Bildquelle: <https://naadispeaks.wordpress.com/2017/11/08/artificial-neural-networks-with-net-in-azure-ml-studio/>

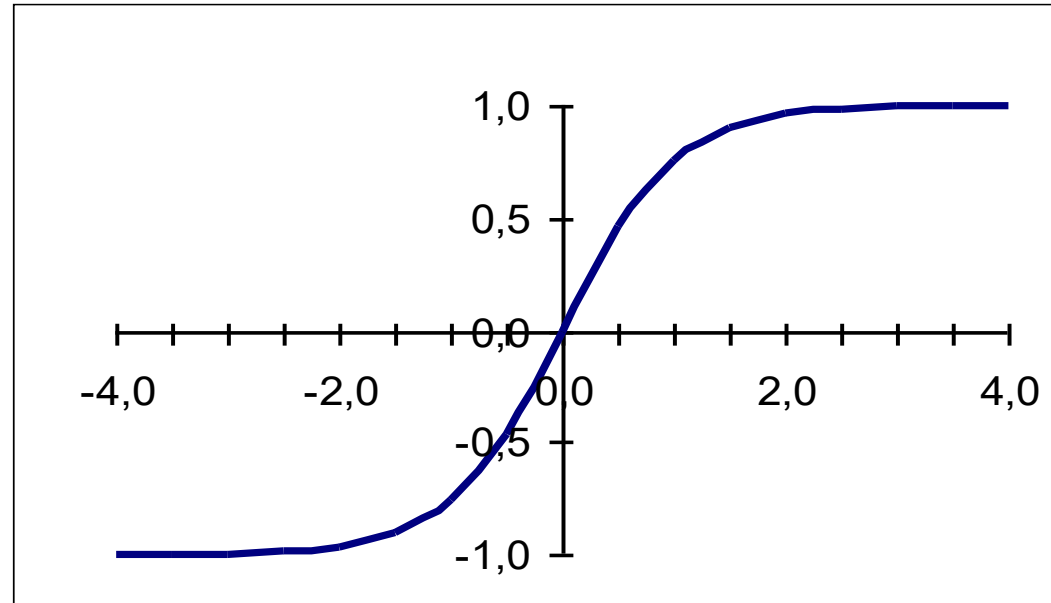
Output Funktionen (1)



$y = 1$, wenn $x \geq \theta$, sonst 0

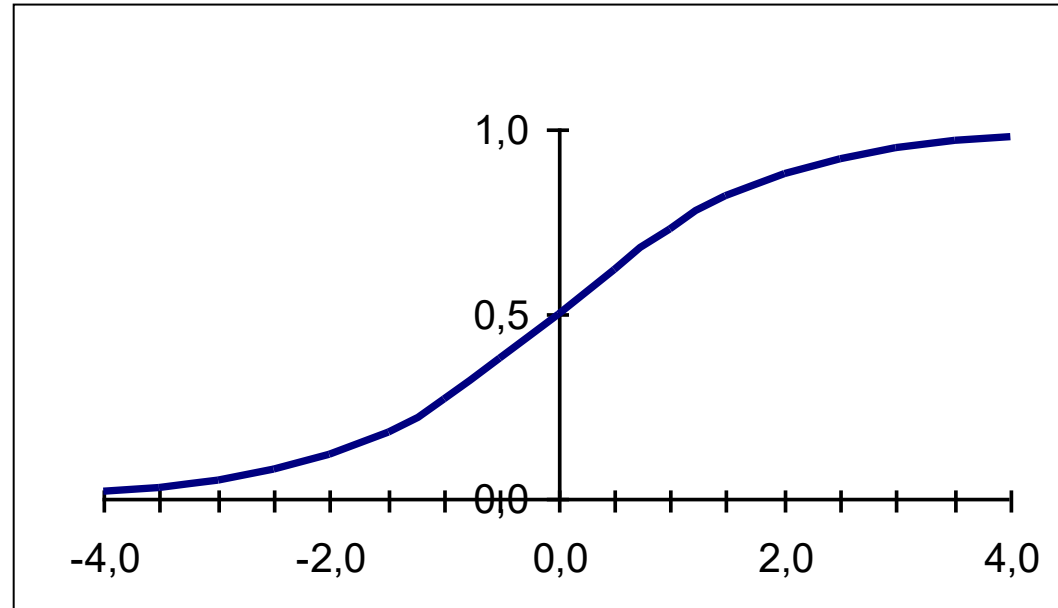
$$\text{mit } x = \sum_{i=1}^n x_i w_i$$

Output Funktionen (2)



$$y = \tanh \left(\sum_{i=1}^n x_i w_i - \theta \right)$$

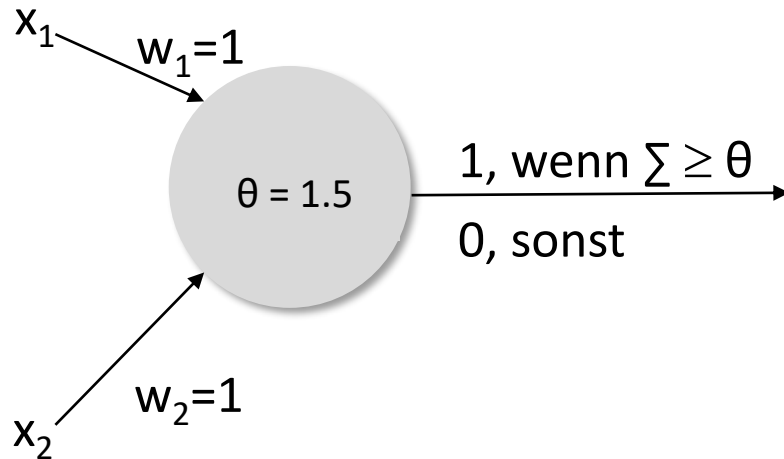
Output Funktionen (3)



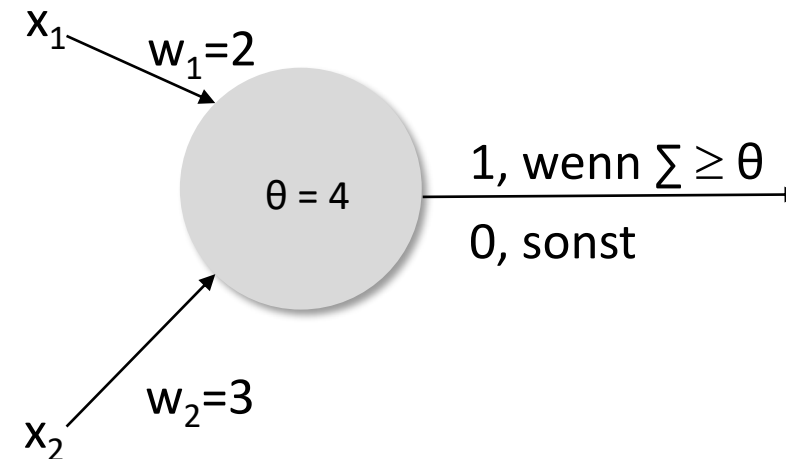
$$y = \frac{1}{1 + e^{-\sum_{i=1}^n x_i w_i - \theta}}$$

TLU Beispiel: logisches UND

AND Neuron: $x_1 \wedge x_2$



x_1	x_2	$w_1 * x_1 + w_2 * x_2$	y
1	1	2	1
1	0	1	0
0	1	1	0
0	0	0	0

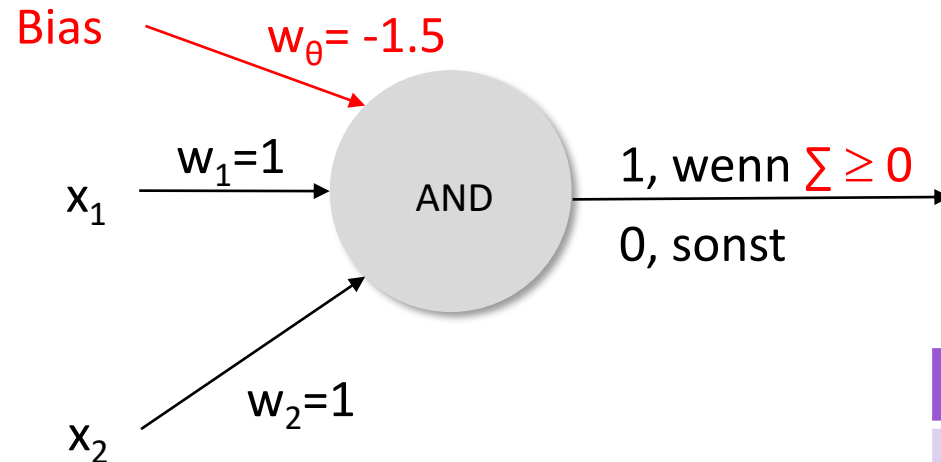


x_1	x_2	$2x_1 + 3x_2$	y
1	1	5	1
1	0	2	0
0	1	3	0
0	0	0	0

Bias

Der Schwellwert kann auch als Gewicht eines Eingangs, der immer mit +1 belegt ist, interpretiert werden. Das wird **Bias** genannt (Bias-Unit), der Schwellwert wird als Gewicht negiert.

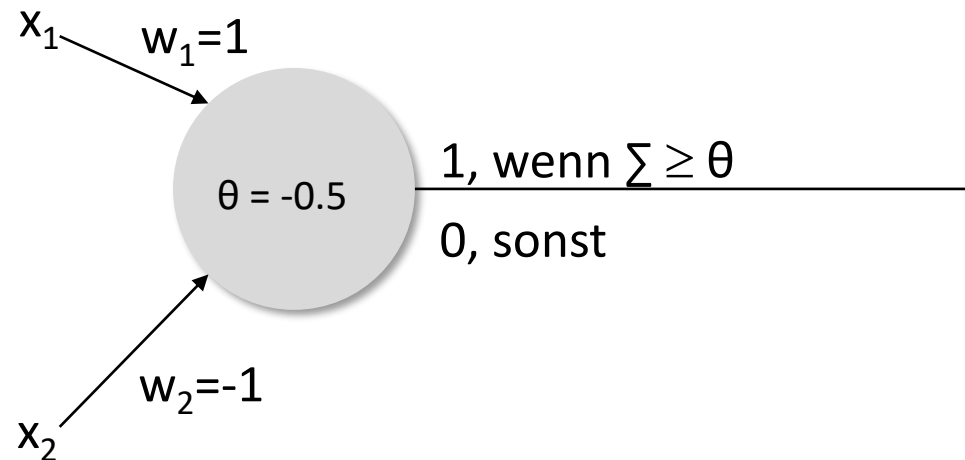
Beispiel: AND Neuron: $x_1 \wedge x_2$



x_1	x_2	Bias	$x_1 + x_2 + \text{Bias}$	y
1	1	-1.5	0.5	1
1	0	-1.5	-0.5	0
0	1	-1.5	-0.5	0
0	0	-1.5	-1.5	0

TLU Beispiel: logische Implikation

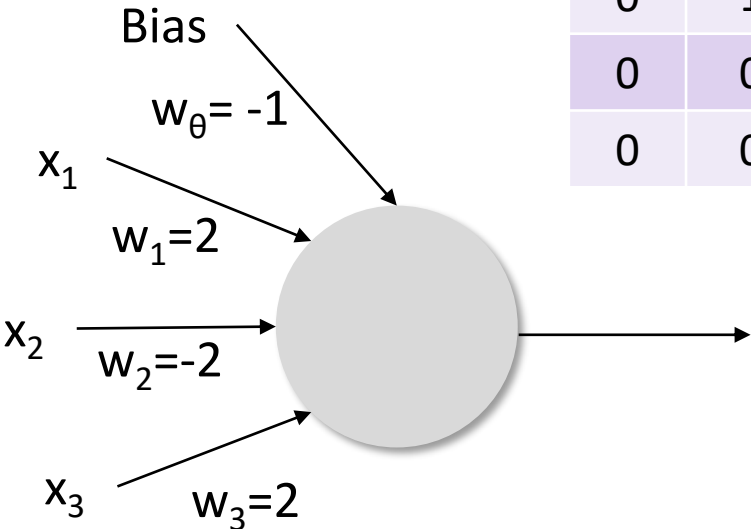
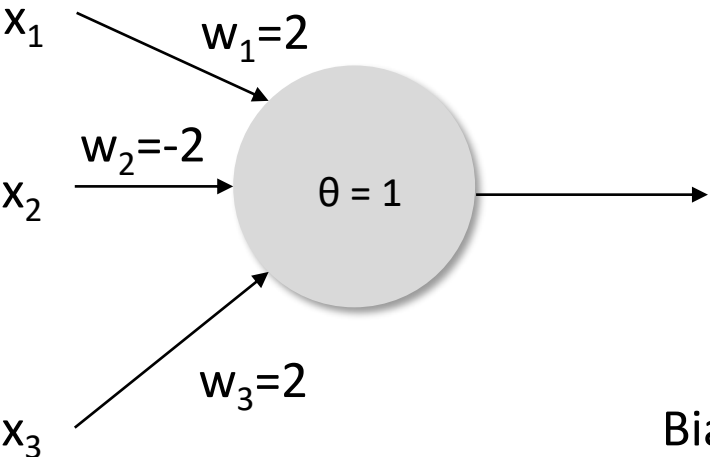
Erstellen Sie ein Neuron für Implikation: $x_2 \Rightarrow x_1$



x_1	x_2	$x_1 - x_2$	y
1	1	0	1
1	0	1	1
0	1	-1	0
0	0	0	1

TLU: Übung

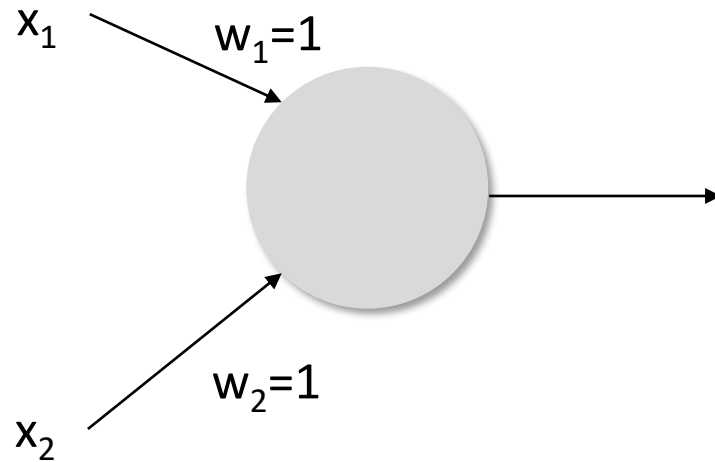
Erstelle ein Neuron für $(x_1 \wedge \neg x_2) \vee (x_1 \wedge x_3) \vee (\neg x_2 \wedge x_3)$



x_1	x_2	x_3	$2x_1 - 2x_2 + 2x_3$	+Bias	y
1	1	1	2	1	1
1	1	0	0	-1	0
1	0	1	4	3	1
1	0	0	2	1	1
0	1	1	0	-1	0
0	1	0	-2	-3	0
0	0	1	2	1	1
0	0	0	0	-1	0

TLU Beispiel: XOR

Erstelle ein Neuron für XOR



x_1	x_2	$x_1 + x_2$	y
1	1	2	0
1	0	1	1
0	1	1	1
0	0	0	0



Was erwartet Dich in diesem Kapitel?

Worüber reden wir hier überhaupt?

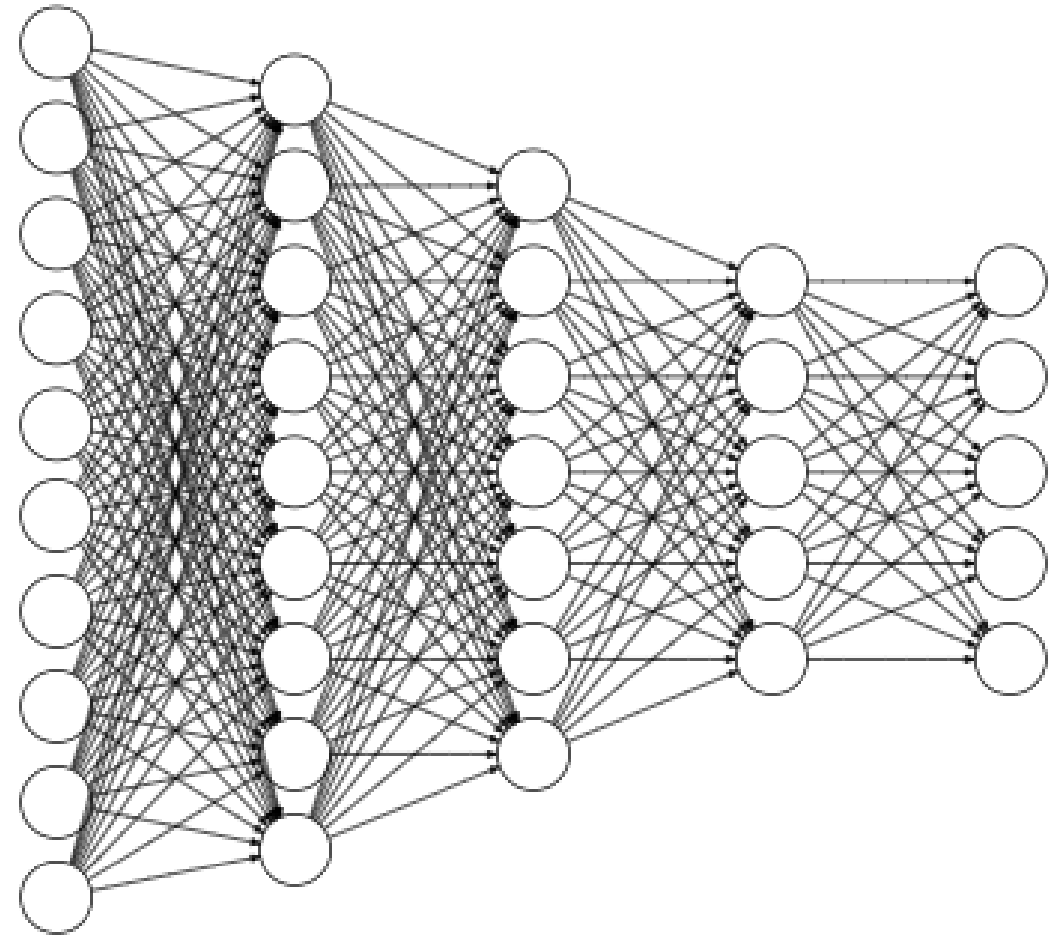
Entstehungsgeschichte

Neuron / Perceptron

Netze

Lernen ...

Tools



Bildquelle: https://chaobin.github.io/images/2016-04-15-neural-network-on-digit-recognition/2016-04-15-neural-network-on-digit-recognition_5_0.svg

Künstliches Neuronales Netz (1)

entsteht durch

- Verknüpfung vieler einfacher Neuronen, die
- über Verbindungen Signale austauschen.

damit ist es

- ein zusammenhängender, gerichteter Graph mit
- gewichteten Kanten,
- Knoten (Neuronen), die mit Werten (Aktivität) versehen sind.

besitzt

- (Eingabe Schicht (input layer))
- Ausgabe Schicht (output layer)
- eventuell verdeckte Schichten (hidden layer)

Künstliches Neuronales Netz (2)

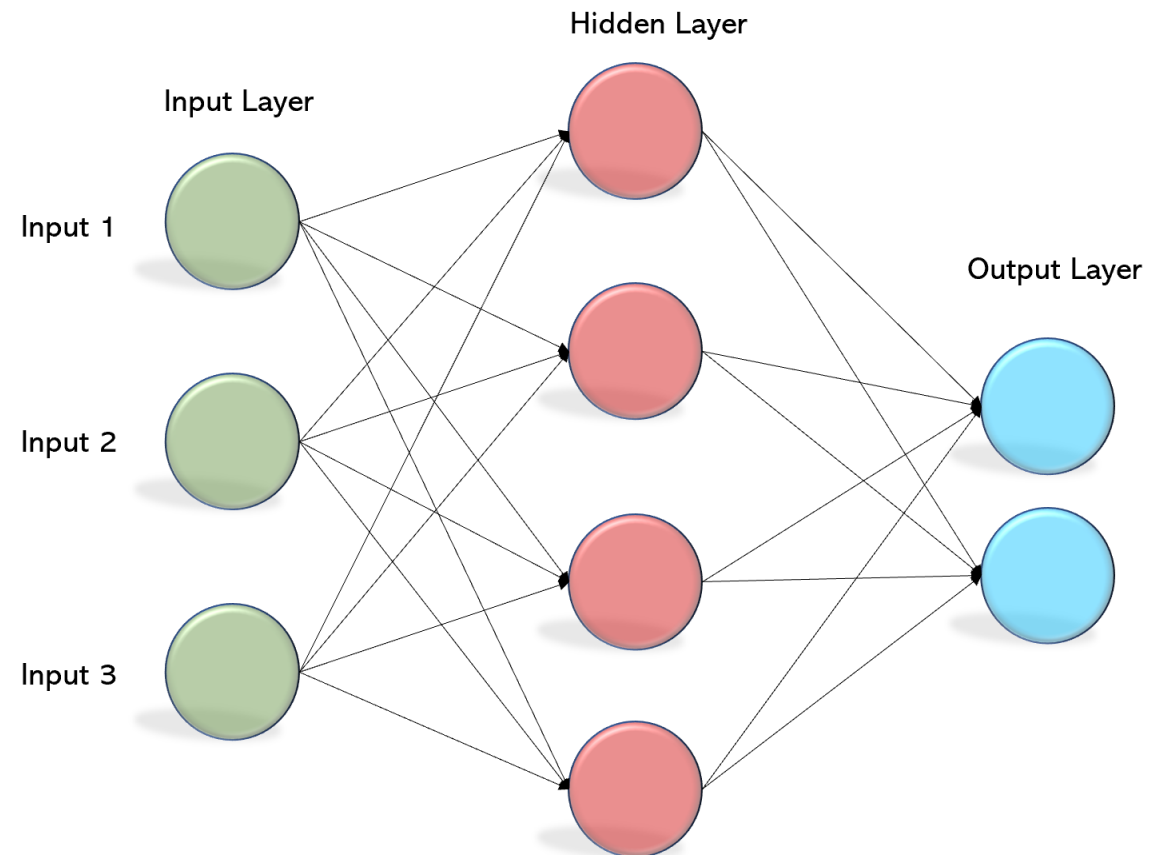
„waagerecht“: Ebene

„senkrecht“: Schicht / Layer

- einschichtig oder
- mehrschichtig (multilayer) (MLP)

Beispiel multilayer:

- **Input Layer**
- ein oder mehrere **Hidden Layer**
- **Output Layer**

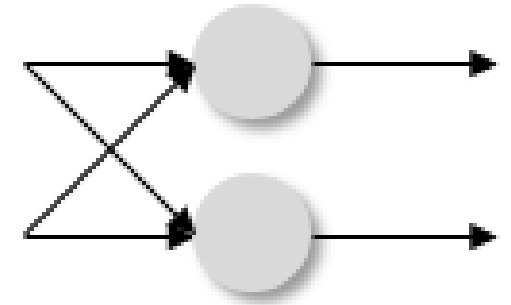


Bildquelle: <https://becominghuman.ai/multi-layer-perceptron-mlp-models-on-real-world-banking-data-f6dd3d7e998f>

Forward Propagation / Feedforward Struktur (1)

Vorwärtsgerichtete Netze (feedforward) zur Forward Propagation

- dienen zur Output-Berechnung
- Neuronenausgabe nur in Verarbeitungsrichtung,
- kann nicht zum Neuron durch eine rekurrente Kante zurückgeführt werden (azyklischer, gerichteter Graph)



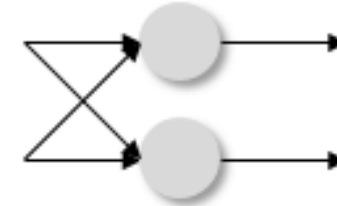
mehrere Möglichkeiten:

- einschichtig
- multilayer
- multilayer mit Abkürzungen

Forward Propagation / Feedforward Struktur (2)

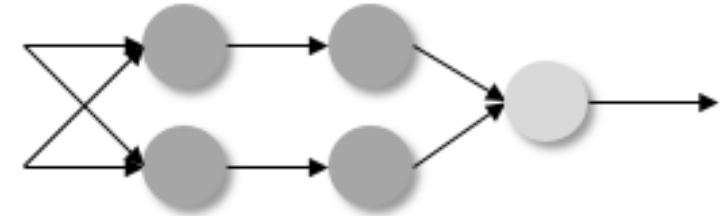
Einschichtiges feedforward-Modell:

- besteht nur aus einer Eingabe- und Ausgabe-Schicht.



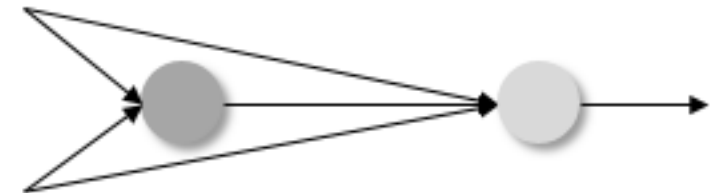
Multilayer feedforward-Modell:

- zwischen Eingabe und Ausgabe-Layer sind verdeckte (hidden) Neuronen(Layer), deren Ausgabe außerhalb des Netzes nicht sichtbar ist,
- ebenenweise verbunden,
- nur Verbindungen zwischen benachbarten Layern.



Mehrschichtiges feedforward-Modell mit Abkürzungen:

- Verbindungen auch zwischen nicht benachbarten Layern (Shortcuts).



Backward Propagation / Rekurrente Struktur (1)

Bei dieser Struktur können Output-Informationen „zurückgeführt“ werden, also als Eingang eines Neurons fungieren.

Rückkopplungen statten das Netz mit einer Art „Gedächtnis“ aus.

Dabei kommt Gradientenberechnung zum Einsatz.

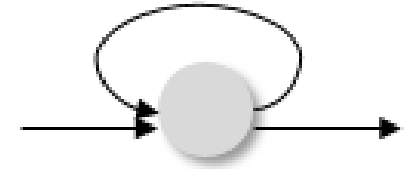
Das Neuron kann

- das gleiche sein (direct feedback)
- in der gleichen Ebene liegen (indirect feedback)
- im gleichen Layer liegen (laterale Rückkopplung).
- Alle Neuronen können komplett miteinander verbunden sein.

Backward Propagation / Rekurrente Struktur (2)

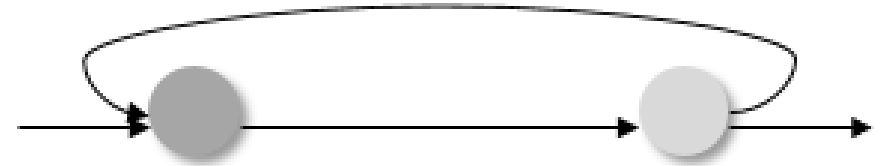
direkte Rückkopplung (direct feedback):

- es gibt rückwärtsgerichtete (rekurrente, feedback loops) Kanten,
- die Rückkopplung kann mit Zeitverzögerung versehen werden,
- dadurch dynamisches Verhalten und „Gedächtnis“,
- ermöglicht die eigene Stärkung oder Hemmung eines Neurons



indirekte Kopplung (indirect feedback):

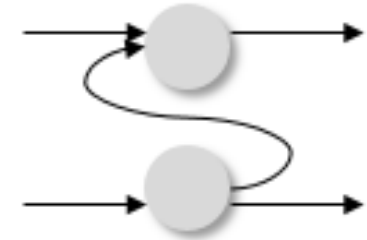
- zwischen Eingabe- und Ausgabe-Schicht ist meist ein mehrschichtiges Netz mit verdeckten (hidden) Neuronen(Layern),
- Rückkopplung über mehrere Layer in der eigenen Ebene,
- Aufmerksamkeit wird auf bestimmte Bereiche gelenkt.



Backward Propagation / Rekurrente Struktur (3)

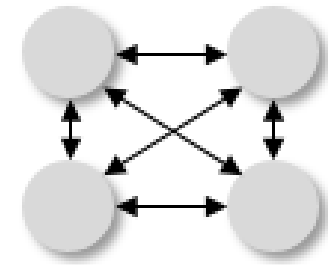
laterale Rückkopplung (winner-takes-all):

- ohne direkte Rückkopplung innerhalb eines Layers,
- Rückmeldung der Information an Neuronen des eigenen Layers,
- Anwendung, falls nur ein Neuron in einem Layer aktiv sein soll.

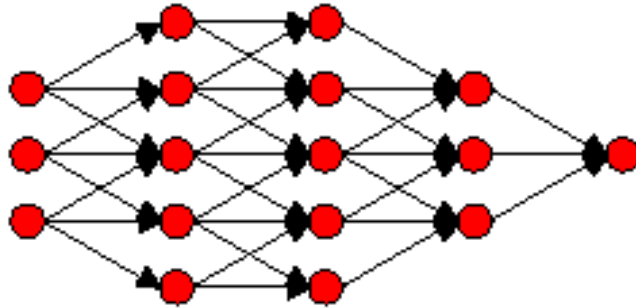


vollständig verbunden:

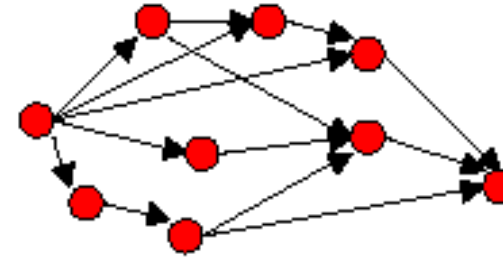
- ohne direkte Rückkopplung,
- symmetrische Verbindungsmatrix (Hopfield-Netze).



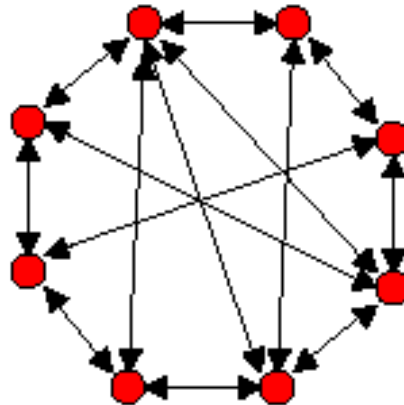
- **Multilayer Model**



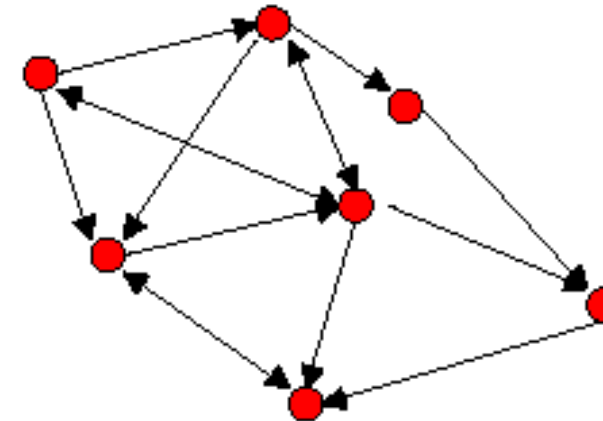
- **Feed-Forward Model**



- **Recurrent (Relaxation) Model, symmetric**



- **Recurrent (Relaxation) Model, asymmetric**



Exkurs in die Graphentheorie ...

Wie das Netz darstellen?

In der *Graphentheorie* werden Graphen mittels

- Adjazenz- (Beziehung Knoten und Knoten) bzw.
- Inzidenzmatrix (Beziehung Knoten und Kanten)

dargestellt.

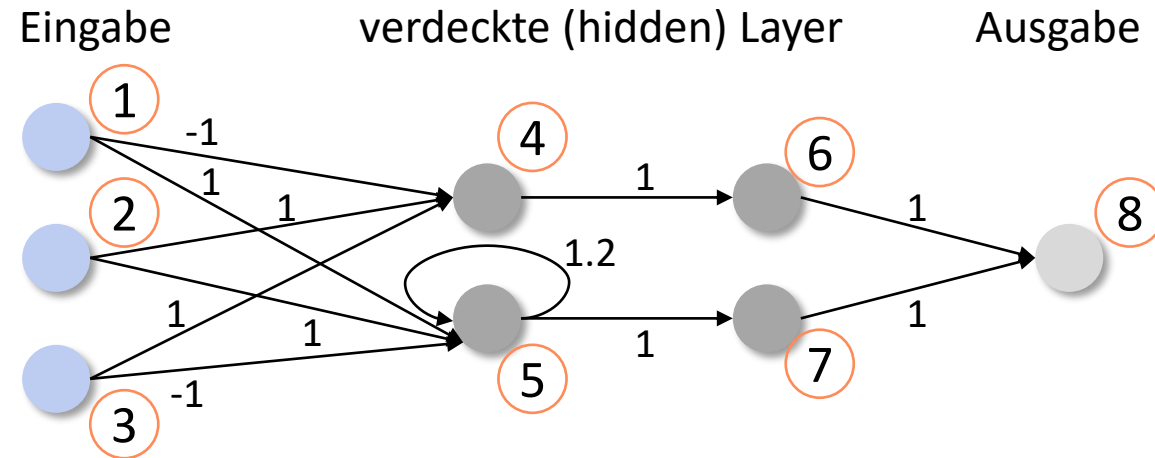
Sehen wir uns die Adjazenzmatrix für unsere KNN genauer an:

- Kennzeichnung von direkten Nachbarschaftsbeziehungen.
- Bei n Knoten erhält man eine $n \times n$ Matrix, in der direkte Knotenverbindungen mit true, andernfalls mit false gekennzeichnet werden.
- Handelt es sich um einen bewerteten/gewichteten Graphen, werden statt true/false die Kosten eingetragen.

KNN: Kosten = Gewichte

statt true/false auch 1/0 oder nur Kennzeichnung der Verbindung.

Beispiel Netzmatrix



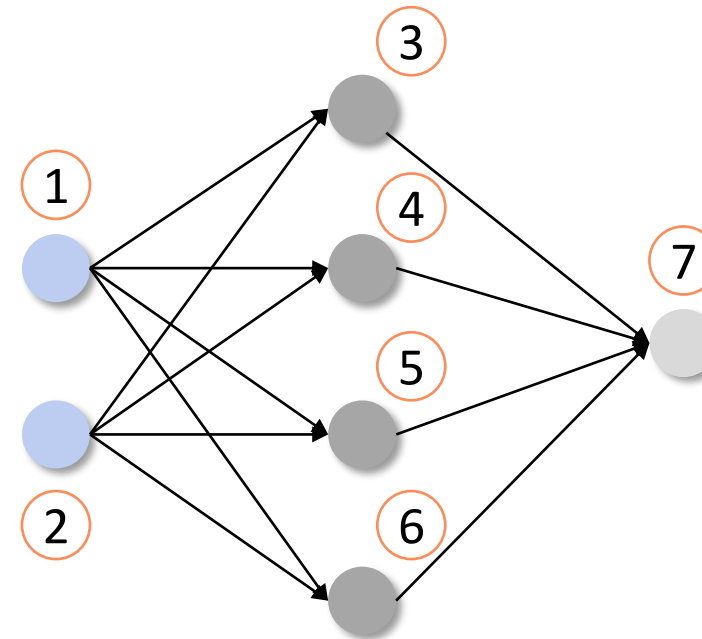
	1	2	3	4	5	6	7	8
1				x	x			
2				x	x			
3				x	x			
4						x		
5					x		x	
6								x
7								x
8								

	1	2	3	4	5	6	7	8
1				-1	1			
2				1	1			
3				1	-1			
4						1		
5					1.2		1	
6								1
7								1
8								

Beispiel vorwärtsgerichtetes Netz (7 Neuronen)

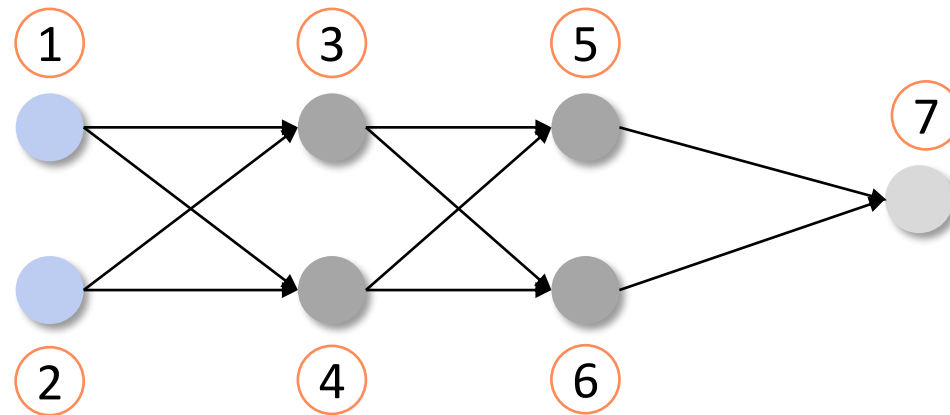
- 2 **Eingabe-Neuronen** (input neuron), die **Eingabeschicht** (input layer) ist die Menge aller Eingabeneuronen. In der Matrix gibt es keine gerichtete Verbindung zu Eingabe-Neuronen
 - 4 **verdeckte Neuronen** (hidden neuron) auf 4 **verdeckten Schichten** (hidden layer). Generell können verdeckte Neuronen auf verschiedenen Schichten verteilt sein.
 - 1 **Ausgabe-Neuron** (output neuron), die **Ausgabeschicht** (output layer).
- In vorwärtsgerichteten Netzen besitzen Ausgabe-Neuronen keine weiterführenden Verbindungen.

	1	2	3	4	5	6	7
1			x	x	x	x	
2			x	x	x	x	
3							x
4							x
5							x
6							x
7							



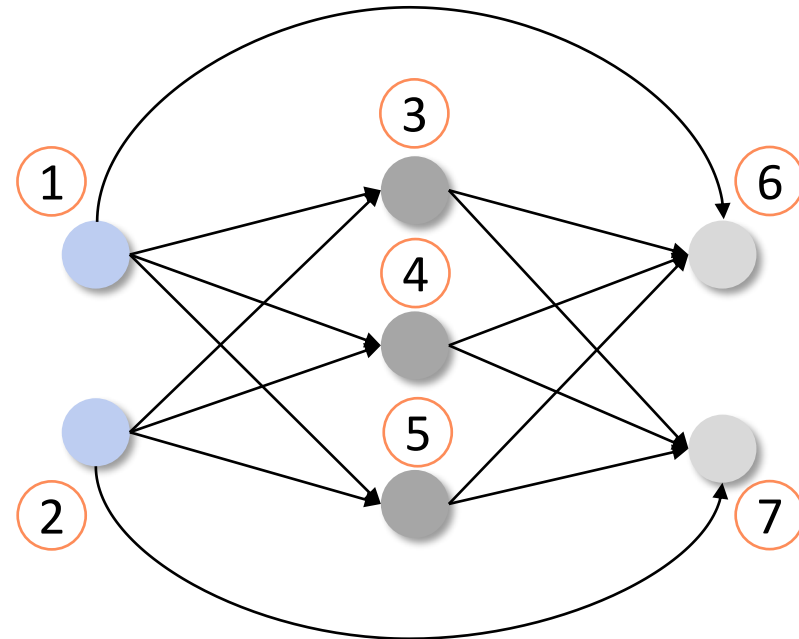
Beispiel vorwärtsgerichtetes Netz mit 2 inneren Layern (7 Neuronen)

	1	2	3	4	5	6	7
1			x	x			
2			x	x			
3					x	x	
4					x	x	
5							x
6							x
7							



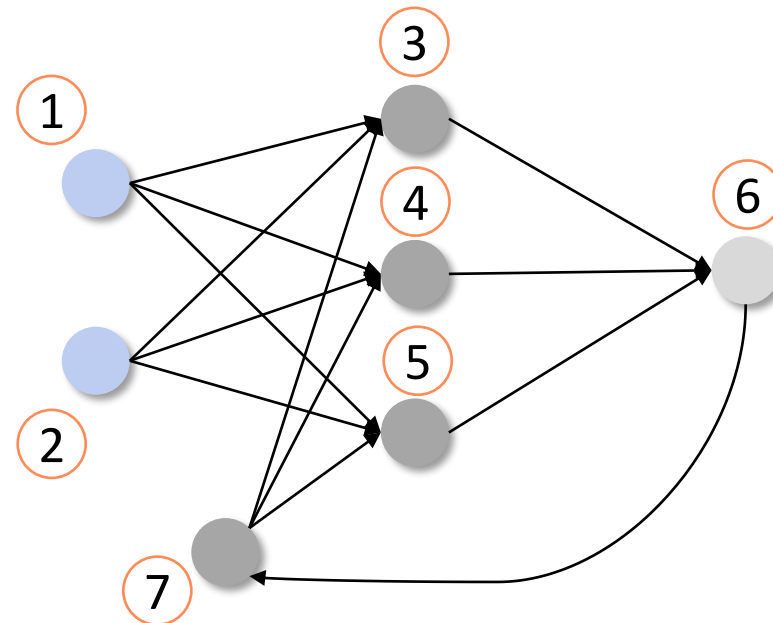
Beispiel: vorwärtsgerichtetes Netz mit Shortcuts (7 Neuronen)

	1	2	3	4	5	6	7
1			x	x	x	x	
2			x	x	x		x
3						x	x
4						x	x
5						x	x
6							
7							



Bsp Netz mit Rückkopplungen über ein spezielles Neuron (7 Neuronen)

	1	2	3	4	5	6	7
1			x	x	x		
2			x	x	x		
3						x	
4						x	
5						x	
6							x
7			x	x	x		



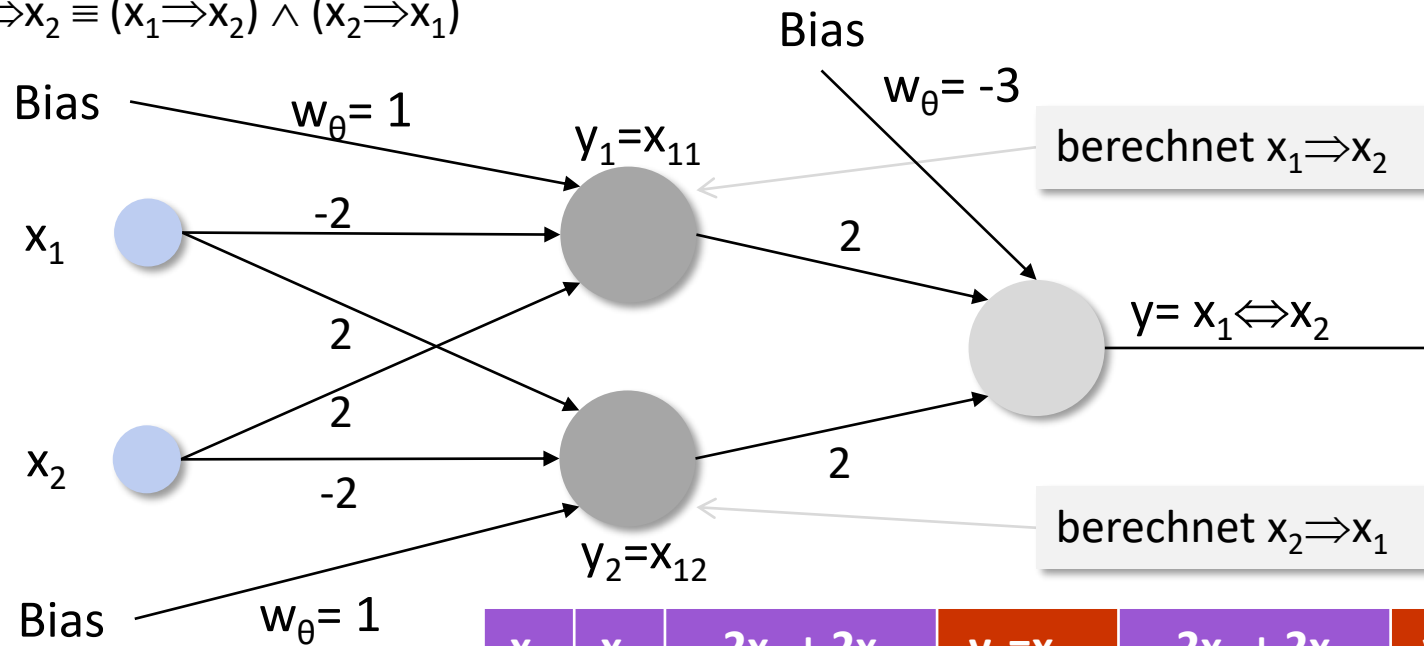
Beispiel: Vollvernetzte Schicht (7 Neuronen)

	1	2	3	4	5	6	7
1		x	x	x	x	x	x
2	x		x	x	x	x	x
3	x	x		x	x	x	x
4	x	x	x		x	x	x
5	x	x	x	x		x	x
6	x	x	x	x	x		x
7	x	x	x	x	x	x	

KNN für Äquivalenz (1)

$$x_1 \Leftrightarrow x_2$$

Idee: $x_1 \Leftrightarrow x_2 \equiv (x_1 \Rightarrow x_2) \wedge (x_2 \Rightarrow x_1)$

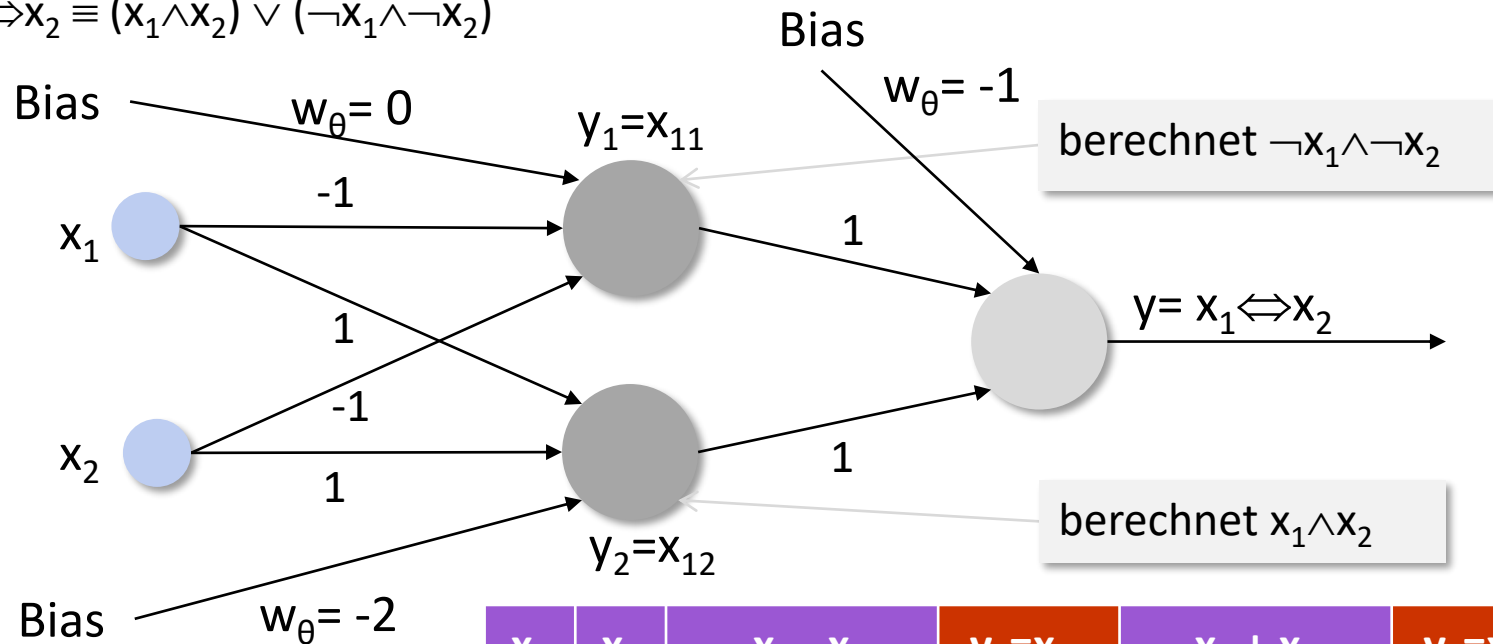


x_1	x_2	$-2x_1 + 2x_2$	$y_1 = x_{11}$	$-2x_2 + 2x_1$	$y_2 = x_{12}$	$2x_{11} + 2x_{12}$	y
1	1	0	1	0	1	4	1
1	0	-2	0	2	1	2	0
0	1	2	1	-2	0	2	0
0	0	0	1	0	1	4	1

KNN für Äquivalenz (2)

$$x_1 \Leftrightarrow x_2$$

$$\text{Idee: } x_1 \Leftrightarrow x_2 \equiv (x_1 \wedge x_2) \vee (\neg x_1 \wedge \neg x_2)$$

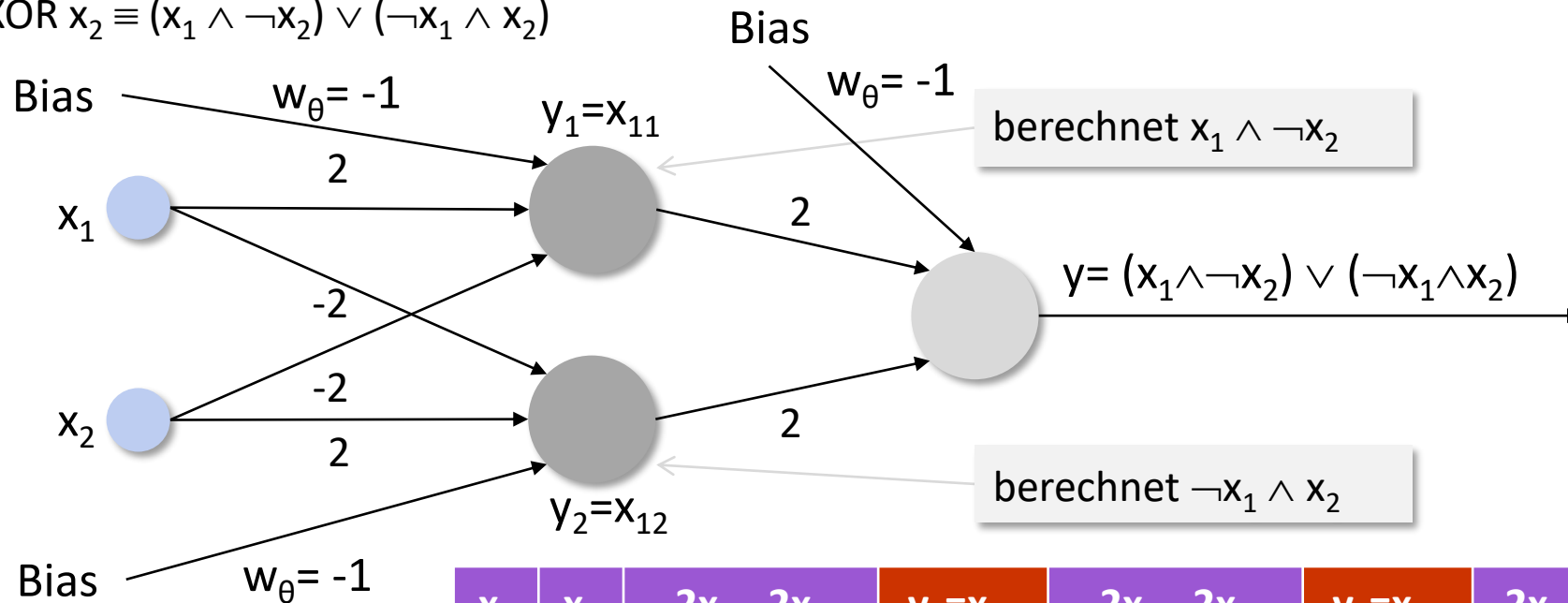


x_1	x_2	$-x_1 - x_2$	$y_1 = x_{11}$	$x_2 + x_1$	$y_2 = x_{12}$	$x_{11} + x_{12}$	y
1	1	-2	0	2	1	1	1
1	0	-1	0	1	0	0	0
0	1	-1	0	1	0	0	0
0	0	0	1	0	0	1	1

zurück zu Beispiel XOR (1)

Erstellen Sie ein KNN für XOR

Idee: $x_1 \text{ XOR } x_2 \equiv (x_1 \wedge \neg x_2) \vee (\neg x_1 \wedge x_2)$



x_1	x_2	$2x_1 - 2x_2$	$y_1 = x_{11}$	$2x_2 - 2x_1$	$y_2 = x_{12}$	$2x_{11} + 2x_{12}$	y
1	1	0	0	0	0	0	0
1	0	2	1	-2	0	2	1
0	1	-2	0	2	1	2	1
0	0	0	0	0	0	0	0

zurück zu Beispiel XOR (2)

XOR ... gehts auch anders?

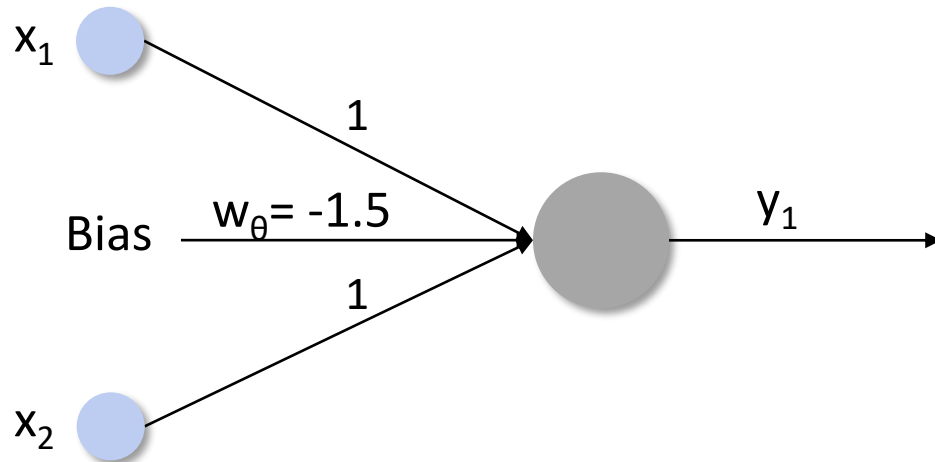
Wo genau lag denn eigentlich das Problem ([Folie](#))?

Idee:

Bei Belegung $x_1=1$ und $x_2=1$ $w_1x_1+w_2x_2=2$ eliminieren

⇒ eigenes Neuron, das nur bei dieser Belegung feuert (UND)

... restliche Berechnung durch anderes Neuron

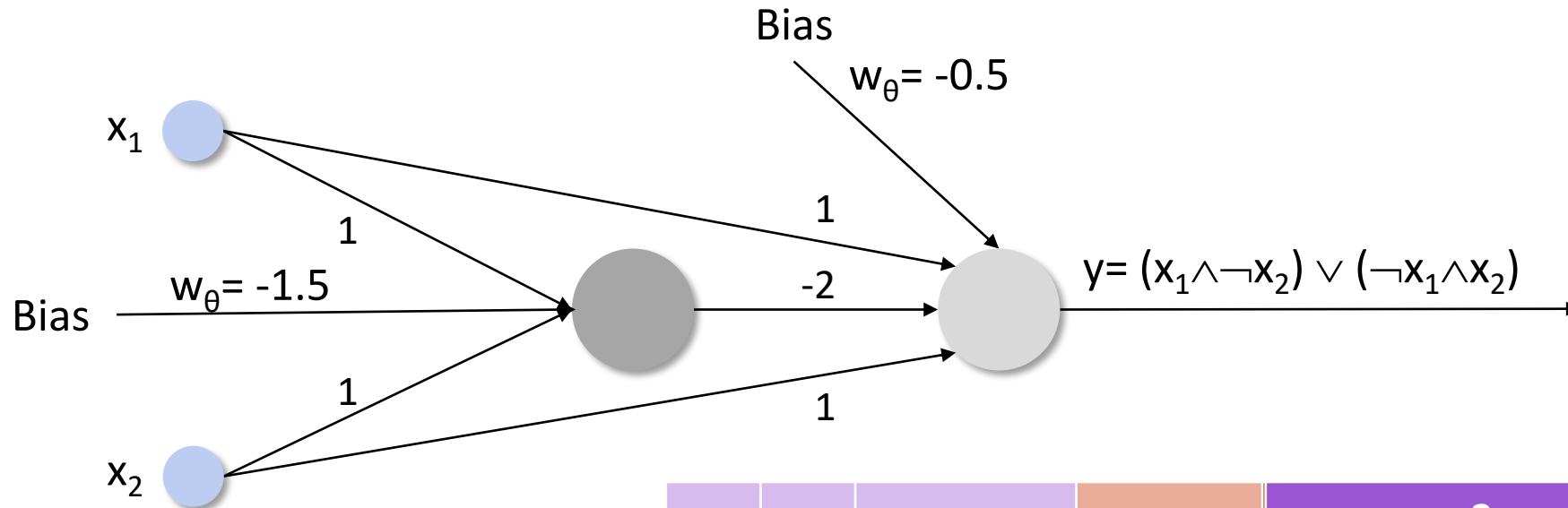


x_1	x_2	$x_1 + x_2$	y
1	1	2	0
1	0	1	1
0	1	1	1
0	0	0	0

x_1	x_2	$x_1 + x_2$	$y_1 = x_3$
1	1	2	1
1	0	1	0
0	1	1	0
0	0	0	0

zurück zu Beispiel XOR (3)

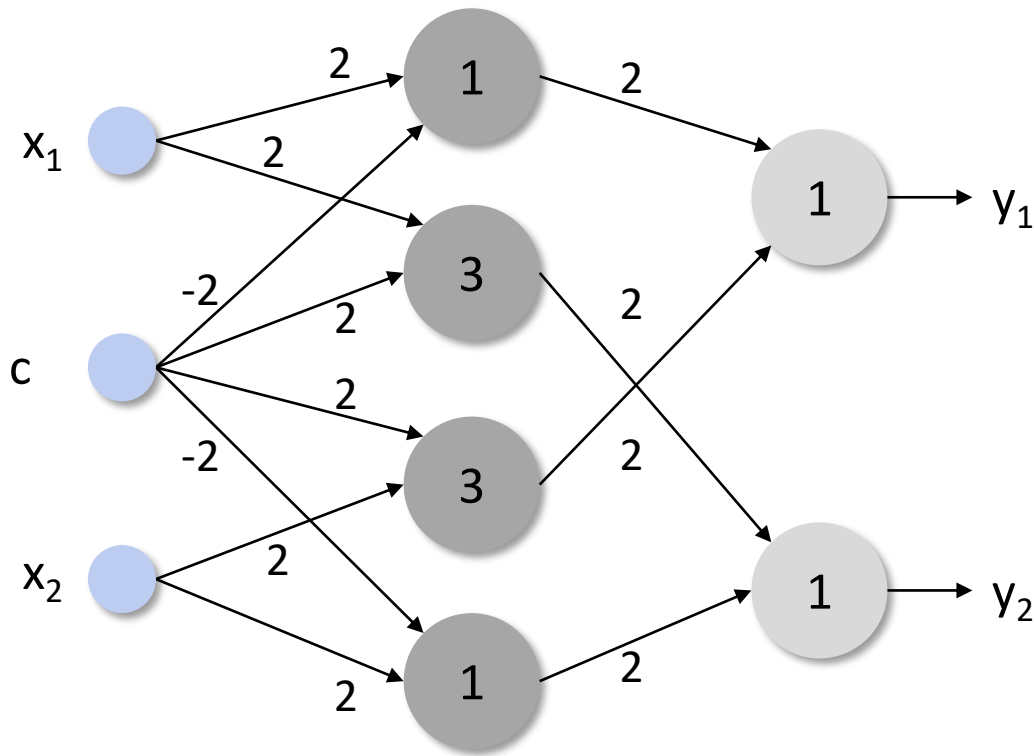
... und nun die restliche Berechnung durch anderes Neuron und den Ausgang des UND-Neurons eliminieren



x_1	x_2	$x_1 + x_2$	$y_1 = x_3$	$x_1 + x_2 - 2x_3$	y
1	1	2	1	0	0
1	0	1	0	1	1
0	1	1	0	1	1
0	0	0	0	0	0

Beispiel: Fredkin-Gatter

Neuronales Netz mit 3 Eingängen und 2 Ausgängen. Einer der 3 Eingänge (c) „entscheidet“, was ausgegeben wird, bei Belegung mit 0 werden die beiden anderen Eingangswerte „durchgereicht“, bei Belegung mit 1 werden die beiden anderen Eingangswerte vertauscht.



c	x_1	x_2	y_1	y_2
1	1	1	1	1
1	1	0	0	1
1	0	1	1	0
1	0	0	0	0
0	1	1	1	1
0	1	0	1	0
0	0	1	0	1
0	0	0	0	0

Was erwartet Dich in diesem Kapitel?

Worüber reden wir hier überhaupt?

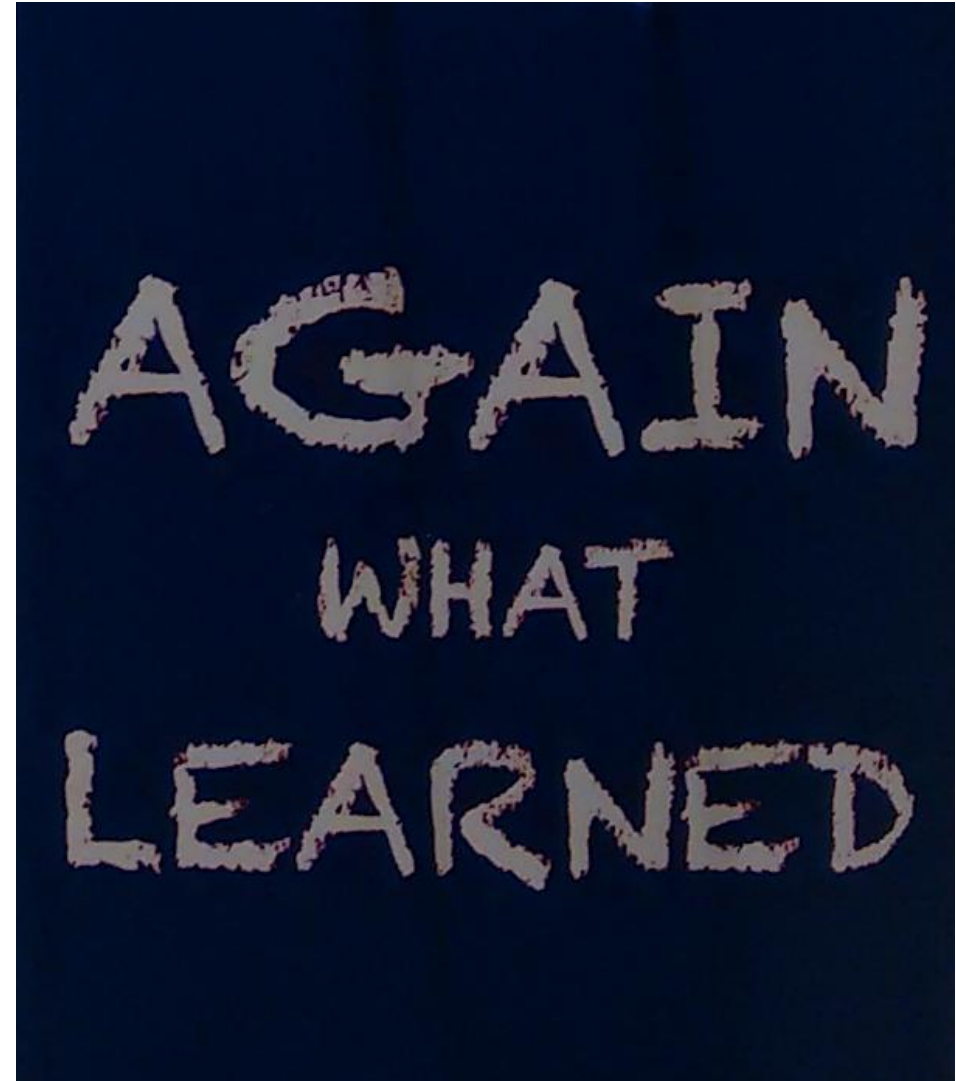
Entstehungsgeschichte

Neuron / Perceptron

Netze

Lernen ...

Tools



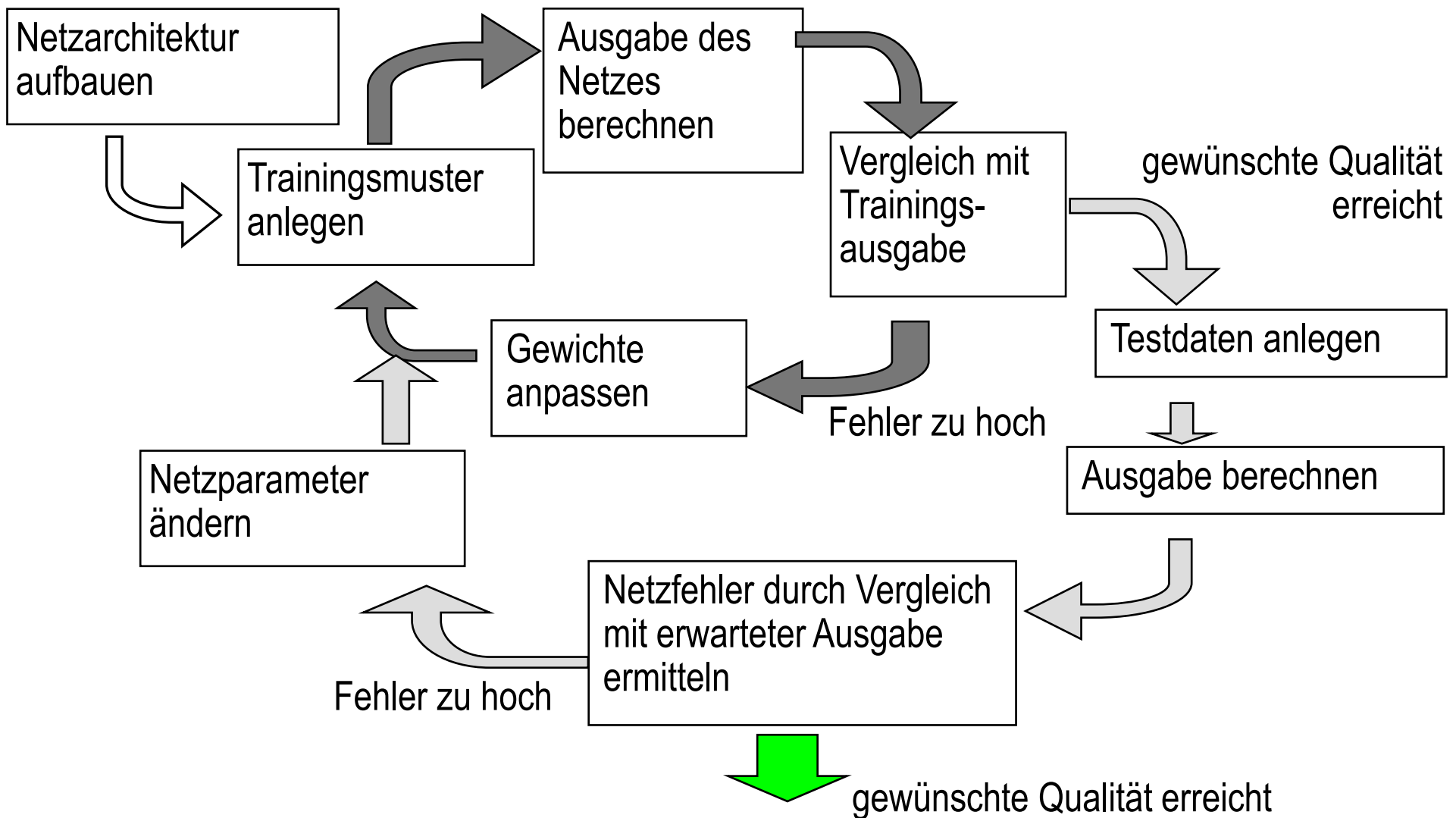
Maschinelles Lernen in einem KNN

- Modifikation der Stärke von Verbindungen (Wichtung),
- Entwicklung neuer Verbindungen (bspw. $w \neq 0$),
- Löschen existierender Verbindungen (bspw. $w = 0$),
- Modifikation der Schwellwerte von Neuronen,
- Modifikation der Funktionen,
- Entwicklung neuer Zellen,
- Löschen von Zellen ...

Grundlegende Idee:

- Start mit zufälligen Werten für Gewichte und Schwellwert.
- Ausgabefehler für eine Menge von Trainingsbeispielen bestimmen. Fehler ist eine Funktion der Gewichte und des Schwellwerts.
- Gewichte und Schwellwert so anpassen, dass der Fehler kleiner wird.
- Anpassung wiederholen, bis der Fehler verschwindet.

Lernen ...



Bildquelle: Uwe Lämmel, 2016, S. 199

näher vorgestellte Lernregeln

Perceptron-Konvergenz-Theorem (Rosenblatt):

Das Perceptron kann in endlicher Zeit alles lernen, was es repräsentieren kann.

- Hebbische Lernregel
Wenn Zelle j eine Eingabe von Zelle i erhält und beide gleichzeitig stark aktiviert sind, dann erhöhe das Verbindungsgewicht w_{ij} .
- Delta Regel (Widrow-Hoff-Regel)
Gewichtsänderung proportional zur Differenz aus erwarteter und tatsächlicher Aktivierung.
- Backpropagation Regel
Verallgemeinerung der Delta-Regel für Netze mit mehr als einer trainierbaren Schicht.

Hebbsche Lernregel (1)

McCulloch-Pitts-Neuronen haben noch keine Lernfähigkeit, das ist aber Hauptmerkmal neuronaler Netze.

Hebbsche Lernregel:

"Wenn ein Axon der Zelle A nahe genug ist, um eine Zelle B zu erregen und wiederholt oder dauerhaft sich am Feuern beteiligt, geschieht ein Wachstumsprozess oder metabolische Änderung in einer oder beiden Zellen dergestalt, dass A's Effizienz als eine der auf B ... feuernden Zellen anwächst."

mathematisch:

Wenn Zelle j eine Eingabe von Zelle i erhält und beide gleichzeitig stark aktiviert sind, dann erhöhe das Gewicht $w_{i,j}$ (d. h. verstärke die Verbindung von Zelle i zu Zelle j).

$$\Delta w_{ij} = \eta \cdot o_i \cdot a_j \text{ und } w_{ij}(t+1) = w_{ij}(t) + \Delta w_{ij}$$

- Δw_{ij} : Änderung des Gewichtes w_{ij}
- η : Konstante (Lernrate)
- o_i : Ausgabe der Vorgängerzelle i
- a_j : Aktivität der Nachfolgerzelle j

Hebbsche Lernregel (2)

Die Gewichtsänderung einer Verbindung zweier Neuronen ist somit bei der Hebbschen Lernregel abhängig von der konstanten Lernrate, der Ausgabe des Vorgängerneurons und dem aktuellen Wert der Aktivitätsfunktion des Nachfolgerneurons.

Es gibt allerdings bei der Hebbschen Lernregel ein Problem: Bei anhaltenden Aktivitäten beider Zellen i und j wachsen die Gewichte ins Unendliche; die Zellen kennen kein "Vergessen". Somit ist die Hebbsche Lernregel nicht realistisch. Durch entsprechende Modifikationen lässt sich dieser Nachteil jedoch beseitigen.

Viele der in den folgenden Jahren entwickelten Lernregeln basieren auf der Hebbschen Lernregel.

Delta Regel

auch Perceptron-Lernregel, Widrow-Hoff-Regel, LMS-Algorithmus

- nur bei **einschichtigen künstlichen Neuronalen Netzen** anwendbar,
- beruht auf Vergleich zwischen gewünschtem und tatsächlichem Output eines Neurons der Ausgabeschicht (o_i).
 $\delta = o_i(\text{gewünscht}) - o_i(\text{beobachtet})$
- **beobachtete Aktivität ist zu niedrig**
Gewichte zwischen sendenden und empfangendem Neuron erhöhen (falls Ursprungsgewicht ≥ 0) bzw. senken (falls Gewicht < 0)
- **beobachtete Aktivität ist zu groß**
Gewichte zwischen sendenden und empfangendem Neuron senken (falls Ursprungsgewicht ≥ 0) bzw. erhöhen (falls Gewicht < 0)
- **beobachtete und gewünschte Aktivität sind gleich groß**
= gewünschtes Resultat $\rightarrow$ keine Veränderung

$$\Delta w_{ij} = \eta \cdot \delta_i \cdot a_j \text{ und } w_{ij}(t+1) = w_{ij}(t) + \Delta w_{ij}$$

- Δw_{ij} : Änderung des Gewichtes w_{ij}
- η : Konstante (Lernrate)
- δ_i : Diff. Ausgabe der Vorgängerzelle i
- a_j : Aktivität der Nachfolgerzelle j

Backpropagation

auch Backpropagation of Error oder Fehlerrückführung

- Form des überwachten Lernens
- Verfahren wurde 1974 von Paul Werbos u.a. entwickelt aber erst 1986 durch Rumelhart, Hinton und Williams bekannt gemacht.
- Die vorgestellten Lernverfahren funktionieren nicht auf den versteckten Schichten. Demzufolge keine Trainingsvorgaben für die Ausgabe der Neuronen der inneren Schichten.
- Verallgemeinerung der Delta-Regel für mehrstufige Netze.
- Fehler des Netzes über alle Trainingsmuster in Abhängigkeit der Gewichte w_i
- Gesucht ist minimaler Fehler (= Tal der Fehlerkurve(-fläche))
- Spezialfall des Gradientenverfahrens (Verfahren des steilsten Abstiegs) aus der Optimierung und basiert auf dem mittleren quadratischen Fehler.

Fehlerkurve

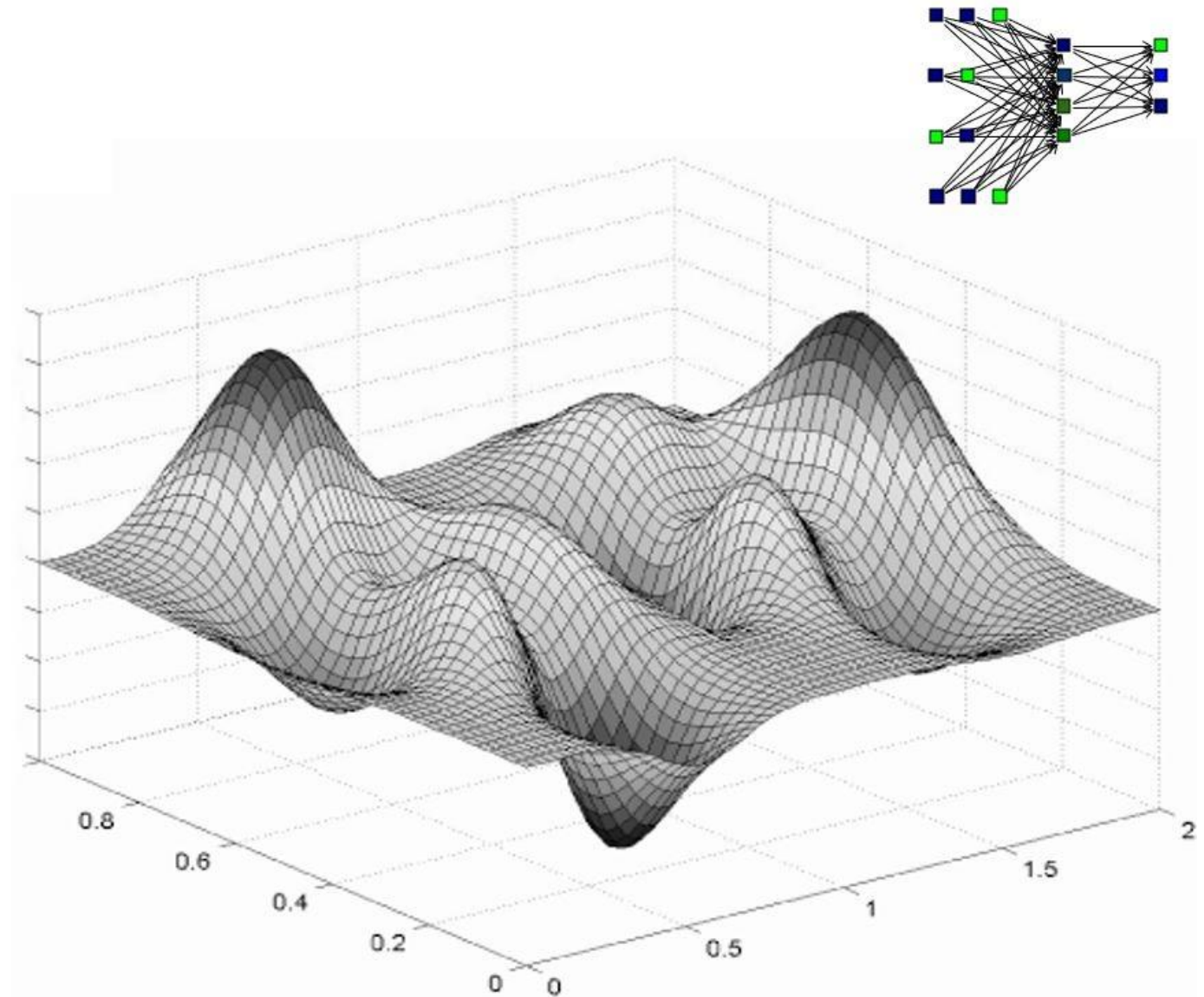
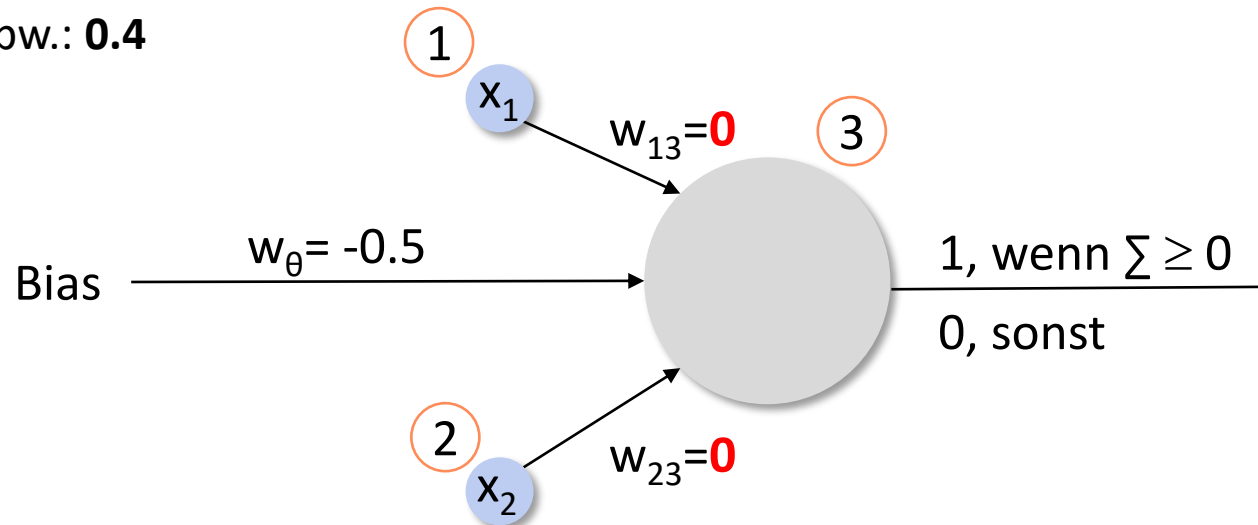


Abbildung aus Uwe Lämmel nn02-forwardNN.ppt

Beispiel: UND-Perceptron (1)

Wir starten mit Gewichten **0** und Schwellwert **0.5**, die Lernrate sollte kleiner als der Schwellwert gewählt werden, bspw.: **0.4**



x1	x2	y gewünscht
1	1	1
1	0	0
0	1	0
0	0	0

1. Iteration:

Eingabemuster $x_1 = x_2 = 1$ wird angelegt. $\sum = 1 \cdot 0 + 1 \cdot 0 - 0.5 = -0.5$
Da $\sum < 0$ ergibt sich $y = 0$.

$$\delta_3 = y_{\text{gewünscht}} - y_{\text{beobachtet}} = 1 - 0 = 1$$

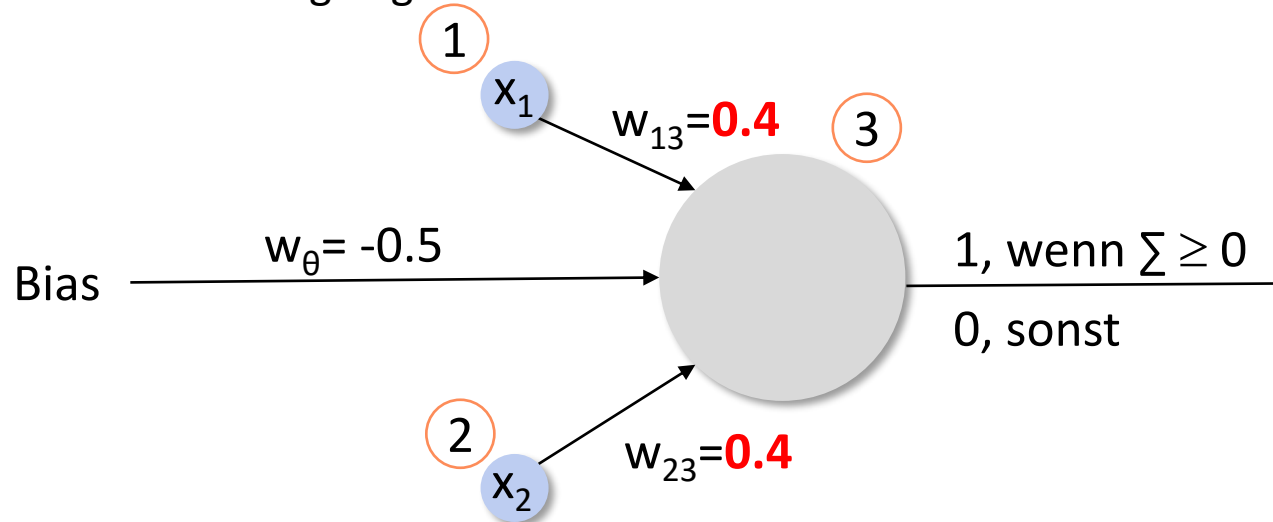
$$\Delta w_{13} = \eta * \delta_3 * x_1 = 0.4 * 1 * 1 = 0.4 \text{ also } w_{13}(1) = w_{13}(0) + \Delta w_{13} = 0.4$$

$$\Delta w_{23} = \eta * \delta_3 * x_2 = 0.4 * 1 * 1 = 0.4 \text{ also } w_{23}(1) = w_{23}(0) + \Delta w_{23} = 0.4$$

Beispiel: UND-Perceptron (2)

Wir haben jetzt die Gewichte **0.4** und Schwellwert 0.5, Lernrate 0.4.

Das zweite Muster wird angelegt.



x1	x2	y gewünscht
1	1	1
1	0	0
0	1	0
0	0	0

2. Iteration:

Eingabemuster $x_1 = 1$ $x_2 = 0$ wird angelegt. $\sum = 1 \cdot 0.4 + 0 \cdot 0.4 - 0.5 = -0.1$

Da $\sum < 0$ ergibt sich $y = 0$.

$$\delta_3 = y_{\text{gewünscht}} - y_{\text{beobachtet}} = 0 - 0 = 0$$

Und damit keine Anpassung der Gewichte.



Beispiel: UND-Perceptron (3)

Wir haben jetzt die Gewichte 0.4 und Schwellwert 0.5, Lernrate 0.4

3. Iteration:

Eingabemuster $x_1 = 0$ $x_2 = 1$ wird angelegt. $\sum = 0*0.4 + 1*0.4 - 0.5 = -0.1$

Da $\sum < 0$ ergibt sich $y = 0$. $\delta_3 = y_{\text{gewünscht}} - y_{\text{beobachtet}} = 0 - 0 = 0$



Und damit keine Anpassung der Gewichte.

4. Iteration:

Eingabemuster $x_1 = 0$ $x_2 = 0$ wird angelegt. $\sum = 0*0.4 + 0*0.4 - 0.5 = -0.5$

Da $\sum < 0$ ergibt sich $y = 0$. $\delta_3 = y_{\text{gewünscht}} - y_{\text{beobachtet}} = 0 - 0 = 0$



Und damit keine Anpassung der Gewichte.

5. Iteration:

Eingabemuster $x_1 = 1$ $x_2 = 1$ wird angelegt. $\sum = 1*0.4 + 1*0.4 - 0.5 = 0.3$

Da $\sum > 0$ ergibt sich $y = 1$. $\delta_3 = y_{\text{gewünscht}} - y_{\text{beobachtet}} = 1 - 1 = 0$



Und damit keine Anpassung der Gewichte.

x1	x2	y gewünscht
1	1	1
1	0	0
0	1	0
0	0	0

x1	x2	y gewünscht
1	1	1
1	0	0
0	1	0
0	0	0

Eine Auswahl von Typen von KNN

Anhand der Struktur und der Propagation können KNN klassifiziert werden.

Das wird an dieser Stelle vernachlässigt.

- Self-Organizing-Maps (SOM) (Kohonen-Netze)
- Growing Neural Gas (GNG)
- Convolutional Neural network (CNN)
- Spiking Neural Networks (SNN)
- Time Delay Neural Network (TDNN)
- Recurrent Neural Network (RNN)
- Hopfield-Netze
- ...

Was erwartet Dich in diesem Kapitel?

Worüber reden wir hier überhaupt?

Entstehungsgeschichte

Neuron / Perceptron

Netze

Lernen ...

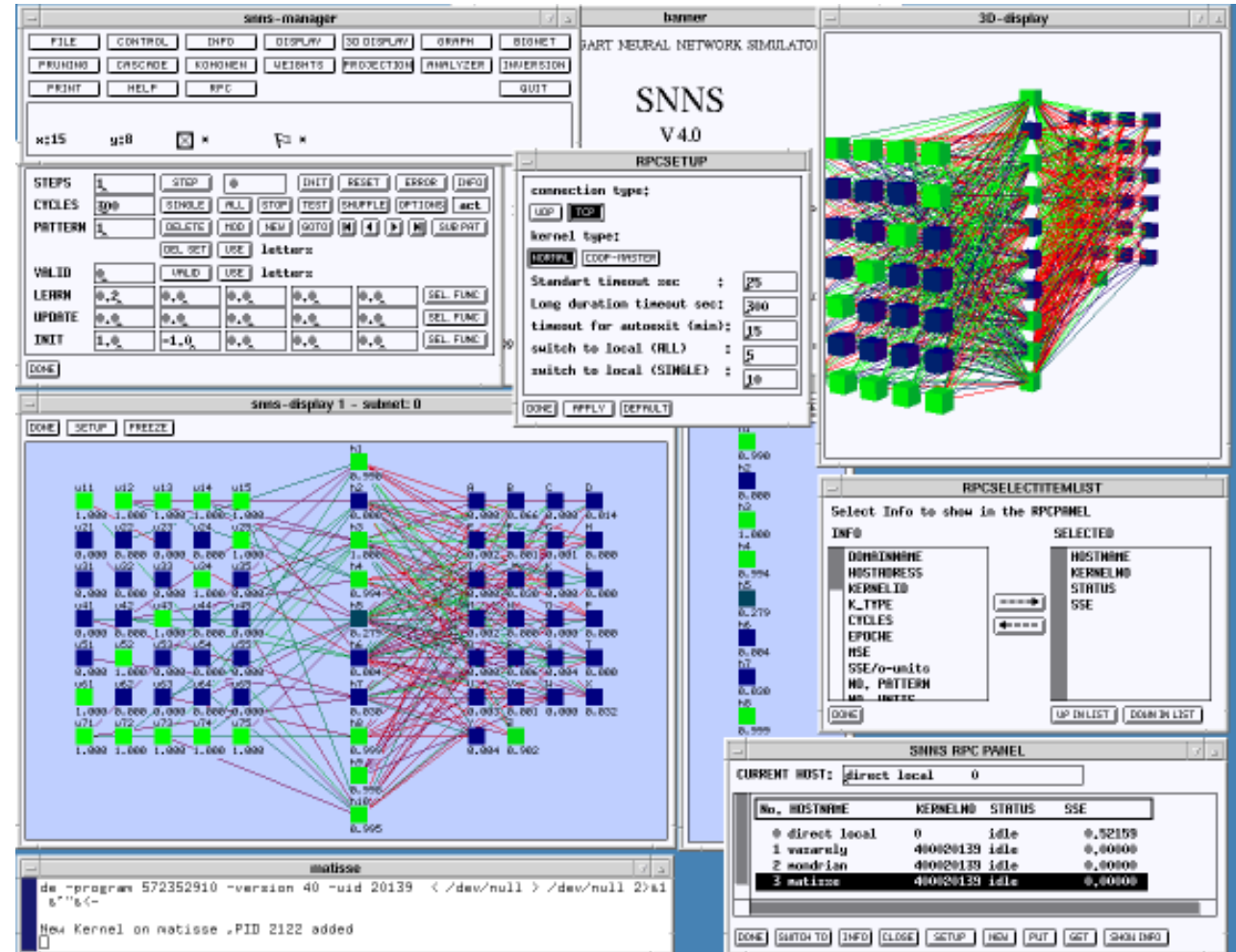
Tools

Implementierungen

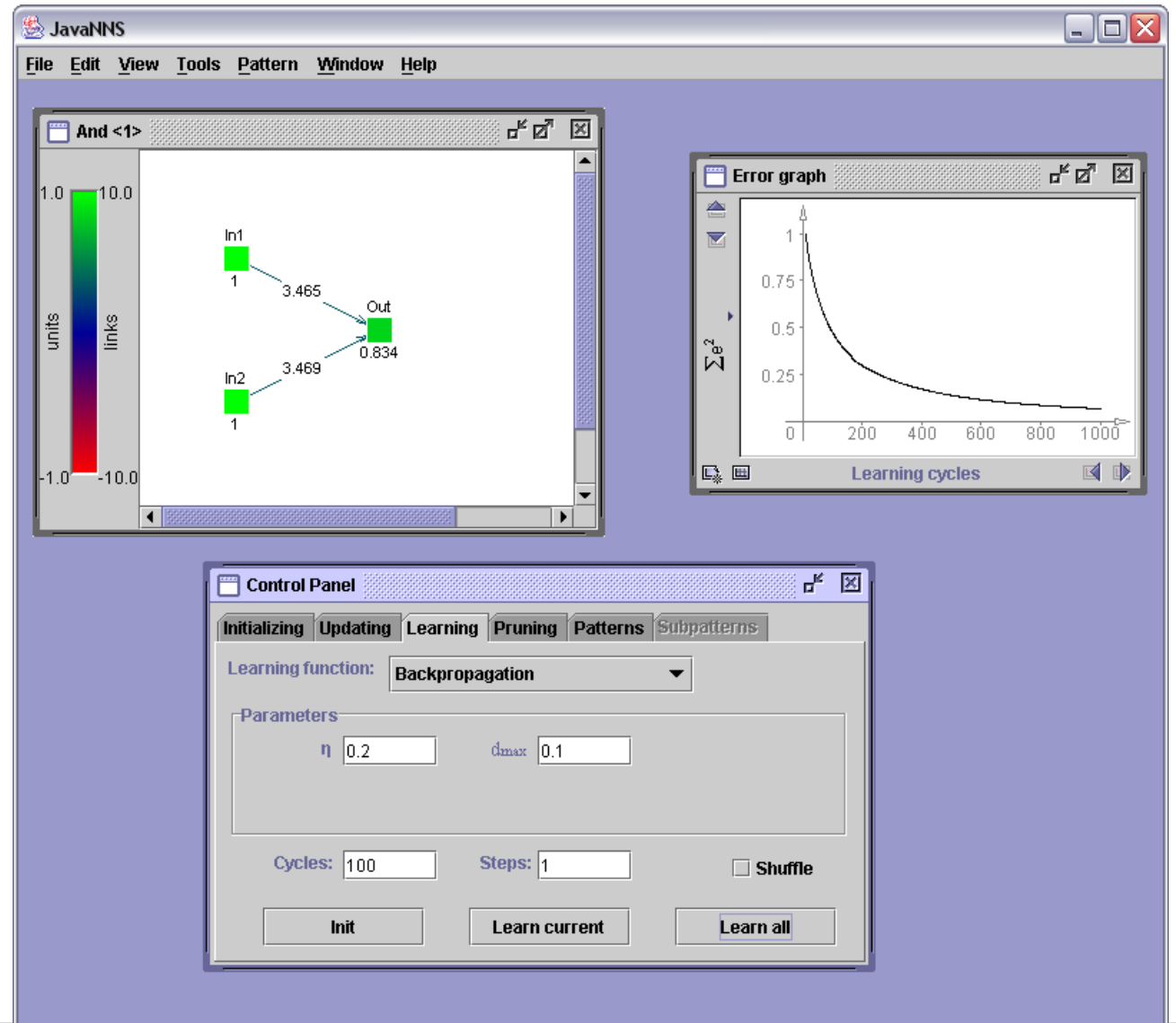
- TensorFlow
- Stuttgarter Neuronale Netze Simulator (SNNS), JavaNNS
www.ra.cs.uni-tuebingen.de/software/JavaNNS/
- EpsiloNN
- OpenNN
- ...

Stuttgarter Neural Network Simulator

www.ra.cs.uni-tuebingen.de/SNNS/

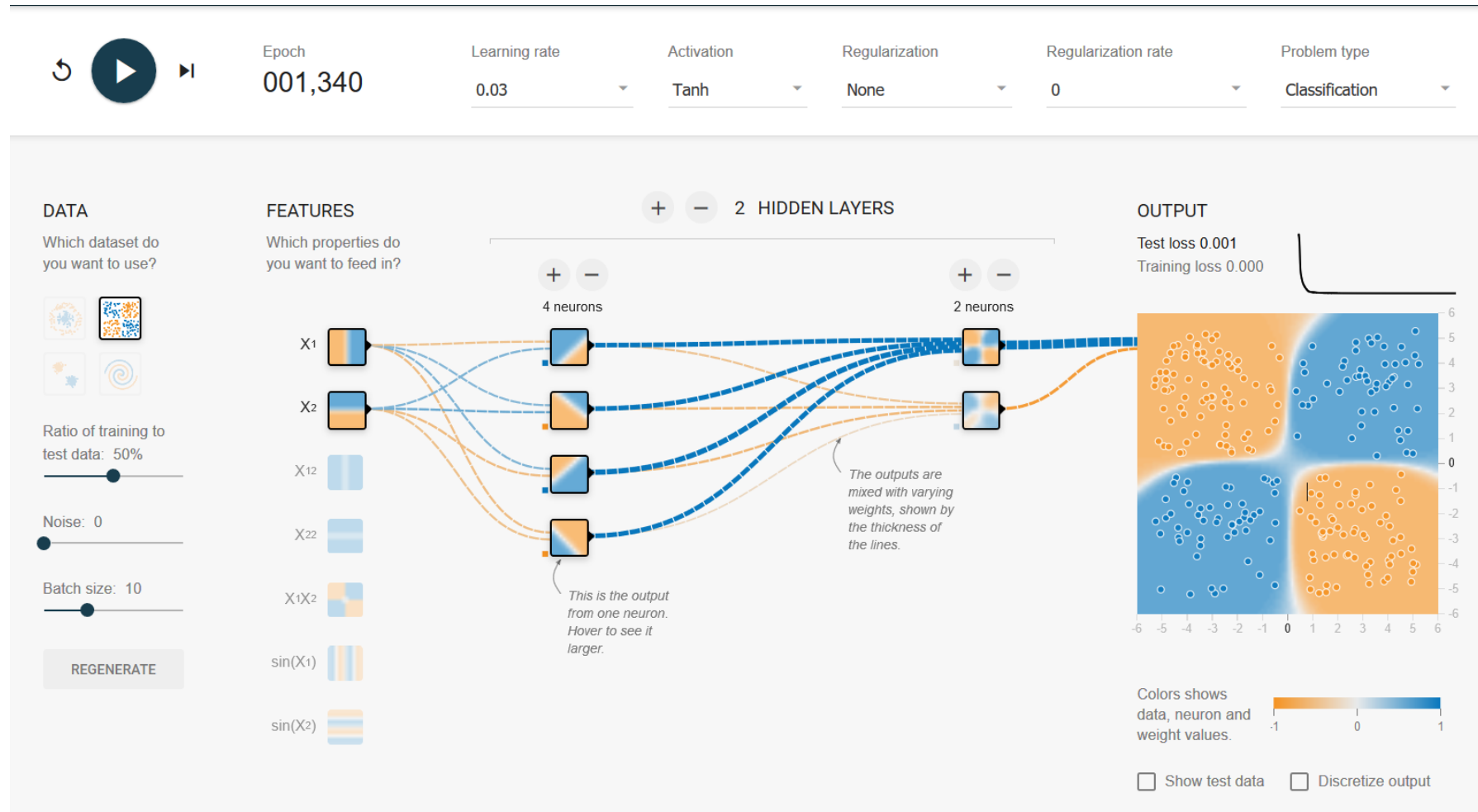


Stuttgarter Neural Network Simulator



TensorFlow

<http://playground.tensorflow.org/>



KNN bauen ... für Klassifikation

- Anzahl der Input Neuronen?
 - Abhängig von der Anzahl der Attribute
 - Abhängig von der Kodierung
- Anzahl der Ausgangsneuronen
 - Abhängig von der Kodierung des Klassenattributes
- Anzahl der Hidden Neuronen?
 - Experimente notwendig
 - generell: nicht mehr als Eingangsneuronen
 - ein Viertel bis Hälfte der Anzahl der Eingangsneuronen kann funktionieren
 - Kapazität des KNN ...

➤ Fehler ermitteln und evaluieren

Experimentelles Verbessern der Ergebnisse

Wie können wir die Ergebnisse verbessern?

- Vorverarbeitung der Daten?
- Architektur des KNN?
- Lernparameter?
- Auswertung der Ergebnisse: Post-Processing?

Dokumentiere Deine Arbeit !!!